



佛山大学
FOSHAN UNIVERSITY



计算机与人工智能学院
School of Computer Science and Artificial Intelligence

Section 6

NLP—关键词提取

Key Words Extraction

任课教师：陈荟慧



引言

- 有一篇很长的文章，我要用计算机提取它的关键词，完全不加以人工干预，请问怎样才能正确做到？
- 关键词提取 (Key phrase extraction, Key words extraction)



目录 CONTENTS



01

关键词提取
技术简介

02

关键词提取
算法

03

自动提取文
本关键词



关键词提取技术简介

- 关键词就是能够反映文本主题或内容的词语。关键词这个概念是随着信息检索学科的出现而被提出，中文关键词是西方信息检索科学移植到中文的直接成果。
- 关键词提取是从单个文本或一个语料库中，根据核心词语的统计和语义分析，选择适当的、能够完整表达主题内容的特征项的过程。
- 关键词提取技术的应用非常广泛，主要应用对象可以分为人类用户和机器用户。在面向读者的应用中，要求所提取的关键词具有很高的可读性、信息性和简约性。
- 关键词提取技术的主要应用有新闻阅读、广告推荐、历史文化研究、论文索引等领域。在NLP中，关键词作为中间产物，应用也非常广泛，主要应用有文本聚类、文本分类、机器翻译、语音识别等领域。





关键词提取技术简介

- 由于关键词具有非常广泛的用途，因此开发出一套实用的关键词提取系统非常重要。这就要求关键词提取算法不仅在理论上正确，更要求在工程上具有很好的实践效果。
- 关键词提取系统的实用性主要表现在以下4个方面。
 - 可读性。一方面，由于中文的字与字之间是没有空格隔开的，需要分词工具对文本进行切分，而分词工具对于专有名词的切分准确率还很低。另一方面，词的表达能力也非常有限，如“市场/经济”，任何一个词“市场”或“经济”都无法表达整个短语的含义。因此，系统所提取出的关键词的可读性对系统的实用性是一个很大的考验。不需要人工标注语料辅进行训练。
 - 高速性。系统应该具有较快的速度，能够及时处理大量的文本。如一个针对各类新闻的关键词提取系统，当新闻产生后，应该能在数秒内提取出该新闻的关键词，才能保证新闻的实时性。
 - 学习性。实用的关键词提取系统，应该能处理非常广泛的领域的文本，而不是仅仅局限于特定领域。随着社会的高速发展，各种未登录词、网络新词频频出现，系统应具有较强的学习能力。
 - 健壮性。系统应该具有处理复杂文本的能力，如中、英文混杂，文本、图表、公式混杂的文本。



关键词提取-讯飞



- <https://www.xfyun.cn/services/keyword-extraction>

为迎接60周年校庆，夯实高水平理工科大学建设的文化基础，学校决定筹建一个以“文化佛山最岭南”为主题的佛山科学技术学院佛山岭南文化展览馆，以反映岭南尤其佛山地区的特色地方文化。

体验版最多100字

开始分析

分析结果

名称	佛山	文化	岭南	理工科	展览馆	校庆	科学技术	夯实	筹建	主题
权重	0.654	0.631	0.631	0.600	0.593	0.589	0.579	0.578	0.575	0.571



豆包

- 古籍文字检索是当前古籍数字化工作的一部分。古籍文字通常存在印刷字模不一致和字体种类多的现象，使用文字的视觉特征实现文字检索是避免字模和字体问题的解决方案。为此，文中提出汉字特征提取编码网络CFEENet (Character Features Extracting and Encoding Network)，使用卷积神经网络提取古籍汉字图像的视觉特征，并使用可训练的通用向量聚合描述层NetVLAD聚合视觉特征并编码，实现数字化古籍的汉字检索。在使用t分布邻域嵌入对CFEENet编码降维后可视化分析，发现CFEENet编码形成的簇密度高、簇间重叠小、编码分辨率高。在多个古籍数据集上测试CFEENet，结果显示单数据集检索精度mAP可以达到83.61%，跨数据集检索精度比现有方法高5.23%，说明CFEENet在古籍汉字检索任务中的适用性与有效性。
- 豆包结果：古籍文字检索、CFEENet、卷积神经网络、NetVLAD、古籍汉字检索精度
- 人工：古籍数字化；局部聚合描述符向量；汉字图像检索；特征编码；



目录 CONTENTS



01

关键词提取
技术简介

02

关键词提取
算法

03

自动提取文
本关键词



关键词提取算法

- 关键词能概括文本的主题，因而帮助读者快速辨别出所选内容是不是感兴趣的内容。目前较常用的无监督关键词提取算法如下。
- TF-IDF算法
- TextRank算法
- 主题模型算法（包括LSA、LSI、LDA等）





TF-IDF算法-词频 (Term Frequency)

- 词频是统计一个词在一篇文档中出现频次的统计量。一个词在一篇文档中出现的频次越高，其对文档的表达能力越强。词频统计量的计算公式如下。

$$tf_{i,j} = \frac{n_{i,j}}{\sum_k n_{k,j}}$$

- 其中， $n_{i,j}$ 表示词 i 在文档 j 中出现的频次， $\sum_k n_{k,j}$ 表示文档 j 的总词数。

$$\text{词频(TF)} = \frac{\text{某个词在文章中的出现次数}}{\text{文章的总词数}}$$





TF (“词频”, Term Frequency)

- 结果是：出现次数最多的词是----“的”、“是”、“在”----这一类最常用的词。它们叫做“停用词” (stop words) 。
- 但是，其它的词是否也有这样的情况？
 - 例如，计算机，语言，编程
- 如何解决这个问题？





TF-IDF算法-逆文档频率

- 逆文档频率是统计一个词出现在文档集中文档频次的统计量。一个词在文档集中越少的出现在文档中，说明这个词对文档的区分能力越强，逆文档频率统计量的计算公式如下所示。

$$\text{idf}_i = \log \frac{|D|}{\left| \{j : t_i \in d_j\} \right| + 1}$$

- 其中， $|D|$ 表示文档集中的总文档数， $|\{j : t_i \in d_j\}|$ 表示文档集中文档 j 出现词 t_i 的文档个数，分母加一是为了避免文档集中没有出现词 t_i ，导致分母为零的情况。





TF-IDF算法

- 词频TF注重词在文档中的出现频次，没有考虑到词在其他文档下的出现频次，缺乏对文档的区分能力。逆文档频率IDF则更注重词的区分能力。
- 两种算法各有不足之处，假设有如下文档：“**在山里，孩子们能享受的快乐只有大山和水，多数时候孩子们都是快乐的，他们的想法都是简单且容易满足的，他们总是期望了解大山外面的世界。**”。
- 文中“孩子们”“快乐”“都是”“他们”“大山”几个词出现次数都是2，文档总词数是60。由逆文档频率统计量公式可知，这几个词语tf值都为0.033，但实际上在这段文本中，“孩子们”“快乐”“大山”这3个词语更为重要。
- 同样地，假设文档集共有2000篇文档，出现“孩子们”“快乐”“都是”“他们”“大山”这几个词的文档数分别为60、30、250、200、20，那么每个词的值分别为1.516、1.810、0.901、0.998、1.979。
- 按照IDF算法计算，“大山”“孩子们”“快乐”比较重要，而“都是”“他们”这类文档中常见的词语，就被赋予较低的idf值。





TF-IDF算法

- 综合权衡词频、逆文档频率两个方面衡量词的重要程度，TF-IDF算法的计算公式如下所示。

$$\text{tf}_{i,j} \times \text{idf}_i = \frac{n_{i,j}}{\sum_k n_{k,j}} \times \log \frac{|D|}{|\{j : t_i \in d_j\}| + 1}$$

- 根据TF-IDF算法计算公式，上列中每个词语的tf值和idf值相乘，得到5个词语的TF-IDF值分别为0.0455、0.0543、0.027、0.0299、0.0594。因此，选取TF-IDF值中相对较大的前3个关键词，即“大山”“孩子们”“快乐”作为这篇文档的关键词。
- 通常，关键词的数量按照TF-IDF值降序排序，选出前几个值较大的作为关键词，也可以根据实际情况确定数量。





- 假设有如下5篇文章，每个汉字视作一个独立的词。
- D1：夜来风雨声，花落知多少。
- D2：人面不知何处去，桃花依旧笑春风。
- D3：春花秋月何时了？往事知多少。
- D4：野火烧不尽，春风吹又生。
- 计算下列字的TF-IDF值（给出公式并化简）。
- D2-“风”：
- D3-“花”：
- D4-“烧”：





TF-IDF算法

- TF-IDF算法倾向于过滤常用的词语，保留相对重要的词语，它实际上只考虑了词的出现频次、出现文档的个数这两个信息，对文本内容的利用程度较低。
- 因此，利用更多的信息进行关键词提取，会对提升关键词提取的效果有很大帮助，如考虑每个词的词性、词的位置信息和出现场合等。
 - 当考虑词的词性时，可以对名词赋予较高的权重，名词往往含有更多的关键信息。
 - 当考虑词的位置时，同样对文本的起始和末尾位置的词赋予较高的权重，始末位置的词往往更为重要。在实际应用中，可以结合应用情况，对算法进行适当的调整，从而达到更好的提取效果。





TF-IDF的一些变体

TF - IDF = 词频(TF) × 逆文档频率 (IDF)

Variants of term frequency (tf) weight

weighting scheme	tf weight
binary	0, 1
raw count	$f_{t,d}$
term frequency	$f_{t,d} / \sum_{t' \in d} f_{t',d}$
log normalization	$\log(1 + f_{t,d})$
double normalization 0.5	$0.5 + 0.5 \cdot \frac{f_{t,d}}{\max_{\{t' \in d\}} f_{t',d}}$
double normalization K	$K + (1 - K) \frac{f_{t,d}}{\max_{\{t' \in d\}} f_{t',d}}$

Variants of inverse document frequency (idf) weight

weighting scheme	idf weight ($n_t = \{d \in D : t \in d\} $)
unary	1
inverse document frequency	$\log \frac{N}{n_t} = -\log \frac{n_t}{N}$
inverse document frequency smooth	$\log \left(\frac{N}{1 + n_t} \right)$
inverse document frequency max	$\log \left(\frac{\max_{\{t' \in d\}} n_{t'}}{1 + n_t} \right)$
probabilistic inverse document frequency	$\log \frac{N - n_t}{n_t}$

Recommended tf-idf weighting schemes

weighting scheme	document term weight	query term weight
1	$f_{t,d} \cdot \log \frac{N}{n_t}$	$\left(0.5 + 0.5 \frac{f_{t,q}}{\max_t f_{t,q}} \right) \cdot \log \frac{N}{n_t}$
2	$1 + \log f_{t,d}$	$\log \left(1 + \frac{N}{n_t} \right)$
3	$(1 + \log f_{t,d}) \cdot \log \frac{N}{n_t}$	$(1 + \log f_{t,q}) \cdot \log \frac{N}{n_t}$





Log的用法

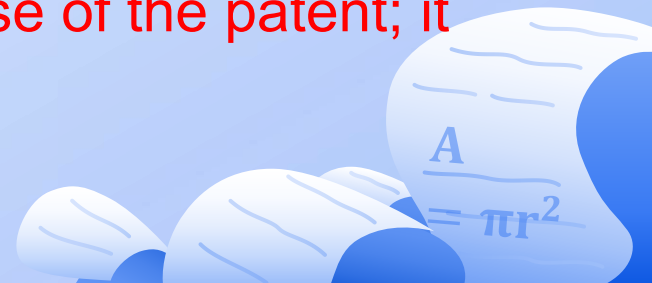
- 在一些数据处理中，经常会把原始数据取对数后进一步处理，原因如下：
 - 1. 缩小数据的绝对数值，方便计算。
 - 2. 取对数后，可以将乘法计算转换称加法计算。例如，信息熵
(<https://blog.csdn.net/taoqick/article/details/72852255>)。
 - 3. 某些情况下，在数据的整个值域中的在不同区间的差异带来的影响不同。例如：某些场景下，200与100的差别可能与2000与1900的差别不同。
 - 4. 取对数之后不会改变数据的性质和相关关系，但压缩了变量的尺度。
- 参考：<https://bbs.pinggu.org/thread-3027640-1-1.html>





TextRank算法

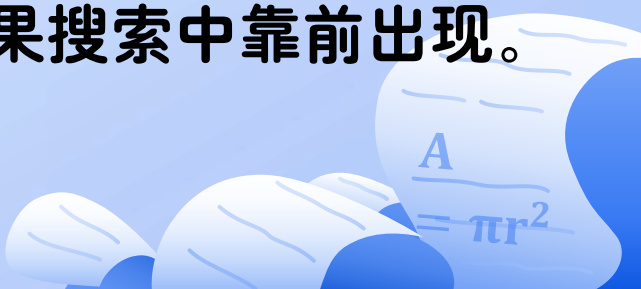
- TextRank算法是一种基于图的文本排序算法，它可以用于自动摘要和提取关键词。
- TextRank算法与TF-IDF算法比较，TextRank算法不同的地方在于，它不需要依靠现有的文档集提取关键词，只需利用局部词汇之间的关系对后续关键词进行排序，随后从文本中提取词或句子，实现提取关键词和自动摘要。
- TextRank算法的基本思想来自Google的PageRank算法。
- The name “PageRank” plays on the name of developer Larry Page (谷歌之父, 创立了谷歌及其母公司Alphabet), as well as of the concept of a web page. The word is a trademark of Google, and the PageRank process has been patented (U.S. Patent 6,285,999). However, the patent is assigned to Stanford University and not to Google. Google has exclusive license rights on the patent from Stanford University. **The university received 1.8 million shares of Google in exchange for use of the patent; it sold the shares in 2005 for \$336 million.**





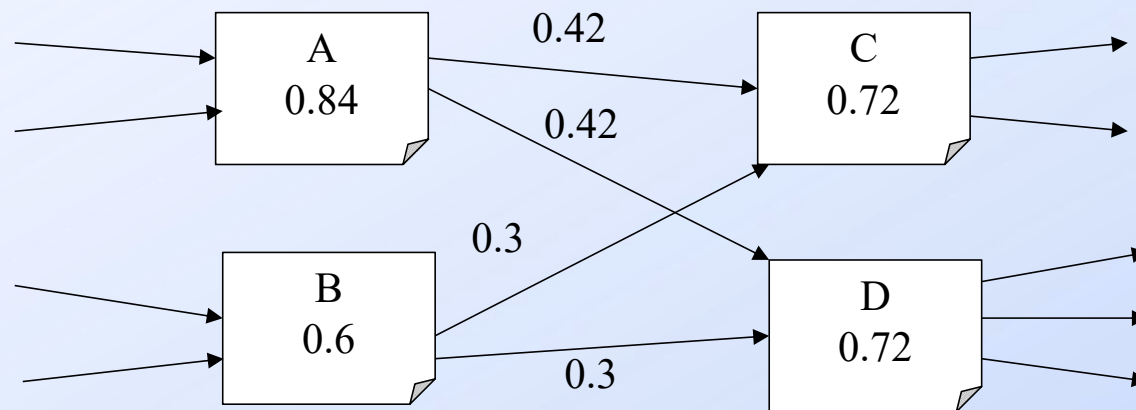
PageRank算法

- PageRank算法最初是在1997年，由Google创始人拉里·佩奇和谢尔盖·布林构建早期的搜索引擎时提出的链接分析算法。最早的搜索引擎是人工对网页进行分类，从而将不同质量的网站区别开。
- 随着互联网的发展，人工分类的方法开始无法处理与日俱增的网站。搜索引擎采用计算用户查询关键词和网页内容的相关性的方法给出搜索结果，但这种方法有其局限性。
- 因此，Google创始人开始研究网页排序问题，PageRank算法由此诞生。该算法是标识网页重要性的一种网页排名算法，也是用于衡量一个站点好坏的标准。通过PageRank算法计算网页的重要性，使更重要的网页在结果搜索中靠前出现。


$$\frac{A}{r} = \pi r^2$$

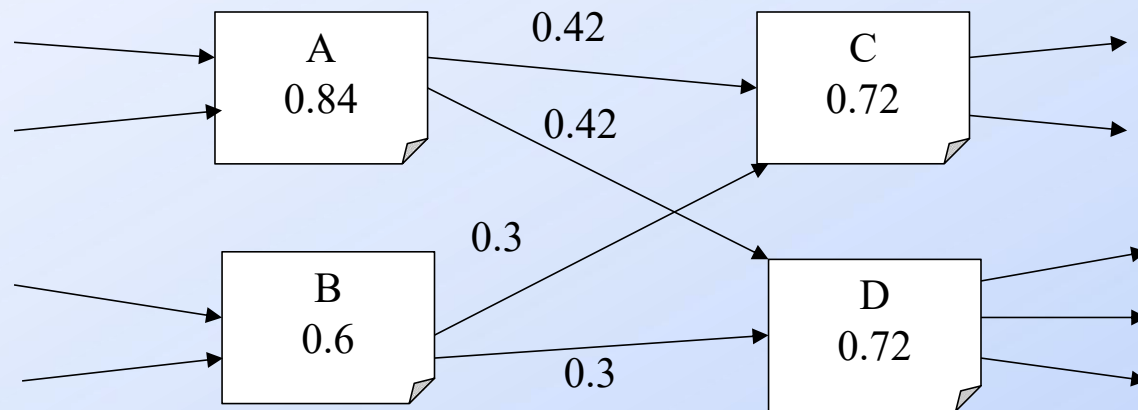
PageRank算法

- 将每一个网页视为一个节点，将网页之间的链接看作有向边，网页之间将构成一张有向图，即网页链接图，如下图所示，网页的得分越高，表示该网页重要程度越高。



PageRank算法

- 网页A自身网页得分为0.84，分别被网页C和网页D链接到，那么网页A会将其得分平均地贡献出去，网页C和D各获得得分0.42。网页B自身得分为0.6，同样将自身得分平均地贡献给链接到的网页，这时网页C和D各获得得分0.3。
- 将网页C和D各自获得的总得分，作为其网页得分。网页C有两个输出链接，网页D有3个，它们将按照链接的个数，再次平均地分配给接下来链接到的网页，以此类推。





PageRank算法

- 要获取网页链接图，通常会遇到以下两个问题。
- **未知上一个网页的初始得分**。在计算网页得分时需要得知上一个链接网页的得分，这时就需要对该方法进行调整。首先，将所有网页的初始得分设置为1，再以迭代的方式求得每个网页的分数，或控制最大迭代次数，最终得到网页的总得分。
- **孤立网页没有得分**。在计算过程中会遇到一些孤立网页，它没有输入链接和输出链接，网页得分将会被计算为0，这时需要对PageRank算法进行调整，加入阻尼系数，使得孤立网页也有网页得分。





PageRank算法

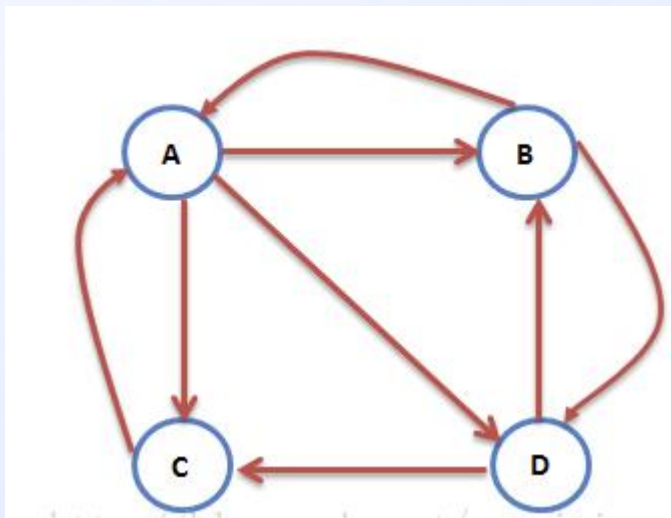
- PageRank算法的基本思想包括两方面内容。一方面，一个网页被越多其他的网页链接，说明这个网页越重要。另一方面，一个网页被越高权值的网页链接，说明这个网页越重要。根据PageRank算法的基本思想，计算一个网页的PageRank值的公式如下所示。

$$S(V_i) = (1-d) + d \times \sum_{j \in \text{In}(V_i)} \left(\frac{1}{|\text{Out}(V_j)|} \times S(V_j) \right)$$

- 其中， $S(v_i)$ 为网页 V_i 的得分， V_j 为链接到网页 V_i 的网页， $\text{In}(V_i)$ 为网页 V_i 的入链集合， $\text{Out}(V_j)$ 为网页 V_j 的出链集合， $|\text{Out}(V_j)|$ 是出链网页的数量， d 为阻尼系数，表示某一网页链接到其他任意网页的概率，取值范围在0~1之间。
- 假设 V_i 是指网页C，那么 $\text{In}(V_i)$ 是指网页A和网页B。每一个网页将所有入链得分加起来，就是网页自身的得分，再将自身的得分平均地分配给每一个出链。


$$A = \pi r^2$$

- 四个网页A B C D
- 初始概率向量为 $(1/4, 1/4, 1/4, 1/4)$



	A	B	C	D
A	0	1/2	1	0
B	1/3	0	0	1/2
C	1/3	0	0	1/2
D	1/3	1/2	0	0

链接转移概率矩阵



一次网页跳转之后，访问到每个网页的概率分布

	A	B	C	D
$V_1 = T \cdot V_0 =$	$\begin{pmatrix} 0 & 1/2 & 1 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{pmatrix} = \begin{pmatrix} 9/24 \\ 5/24 \\ 5/24 \\ 5/24 \end{pmatrix}$	A		
		B		
		C		
		D		

其中， T 为概率转移矩阵。同理，我们可以按照这个方法持续计算到用户第 n 次跳转之后的概率分布，公式如下：

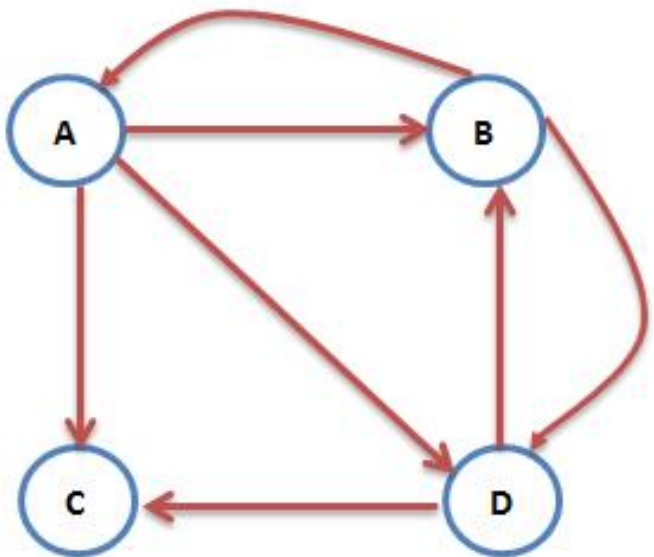
$$V_n = T \cdot V_{n-1} = T^n \cdot V_0$$

终止：当 V_n 不再变化时停止

问题：终止点、陷阱



终止点，如C

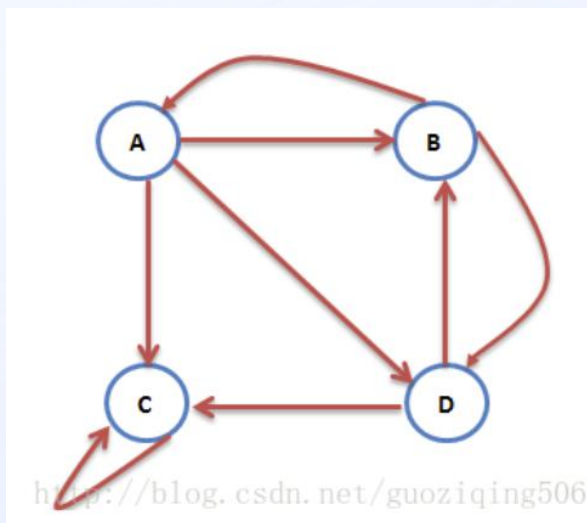


$$T = \begin{pmatrix} 0 & 1/2 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{pmatrix}$$

可见，对应C的第3列全为0，迭代计算PR值，最后的结果都是0：

$$\begin{pmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{pmatrix} \rightarrow \begin{pmatrix} 3/34 \\ 5/24 \\ 5/24 \\ 5/24 \end{pmatrix} \rightarrow \begin{pmatrix} 5/48 \\ 7/48 \\ 7/48 \\ 7/48 \end{pmatrix} \rightarrow \dots \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

陷阱：如C



$$T = \begin{pmatrix} 0 & 1/2 & 0 & 0 \\ 1/3 & 0 & 0 & 1/2 \\ 1/3 & 0 & 1 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{pmatrix}$$

这种情况下，陷阱的PR值为1，而其他正常网页的PR值为0。迭代计算的结果大致如下：

$$\begin{pmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{pmatrix} \rightarrow \begin{pmatrix} 3/24 \\ 5/24 \\ 11/24 \\ 5/24 \end{pmatrix} \rightarrow \begin{pmatrix} 5/48 \\ 7/48 \\ 29/48 \\ 7/48 \end{pmatrix} \rightarrow \dots \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix}$$



修正公式

$$V_n = \alpha T V_{n-1} + (1 - \alpha) V_0$$

当存在终止点时，迭代的结果为：

$$\begin{pmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{pmatrix} \rightarrow \begin{pmatrix} 0.14 \\ 0.21 \\ 0.21 \\ 0.21 \end{pmatrix} \rightarrow \begin{pmatrix} 0.1 \\ 0.1 \\ 0.1 \\ 0.1 \end{pmatrix}$$

当存在陷阱时，迭代的结果为：

$$\begin{pmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{pmatrix} \rightarrow \begin{pmatrix} 0.14 \\ 0.21 \\ 0.42 \\ 0.21 \end{pmatrix} \rightarrow \dots \begin{pmatrix} 0.1 \\ 0.1 \\ 0.7 \\ 0.1 \end{pmatrix}$$



TextRank算法

- TextRank算法通过将文本切分成若干个词或句子建立图模型，采取投票的方式，对文档中的重要成分进行排序，进而实现关键词提取。
- TextRank算法与PageRank的算法思路类似，不同之处在于，TextRank算法实现关键词提取时，需要考虑链接词的重要性和词之间的相似性，它将构成一个加权图，而PageRank算法构成有向无权图。





TextRank算法

- TextRank算法在计算每个词的链接词得分时，通过赋予不同权重的方式分配得分，不再采取PageRank算法平均分配的方式，并且TextRank算法默认所有词之间都存在链接关系。
- 两个词之间的权重越大，相似度则越高。用 $\omega_{i,j}$ 表示权重，其他部分与PageRank算法类似。TextRank关键词提取算法公式如下所示。

$$WS(V_i) = (1 - d) + d \times \sum_{V_j \in In(V_i)} \left(\frac{\omega_{ji}}{\sum_{V_k \in Out(V_j)} \omega_{jk}} \times WS(V_j) \right)$$

- 其中， $WS(v_i)$ 为词 V_i 的得分， V_j 为链接到的 V_i 词， $In(V_i)$ 为词的入链集合， $Out(V_j)$ 为词 V_j 的出链集合， d 为阻尼系数，表示某个词链接到其他任意词的概率，取值范围在0~1之间。

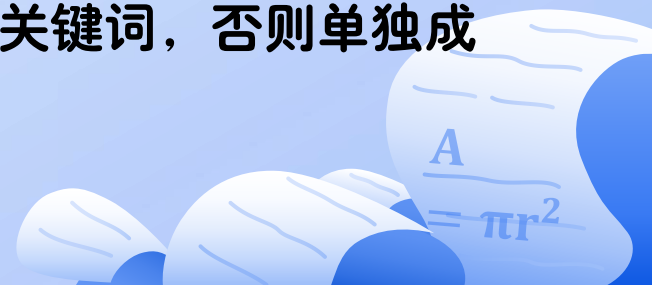




TextRank算法

TextRank关键词提取算法步骤如下。

- 将给定的文本内容 T 依照完整句子切分开，即 $T = \{S_1, S_2, \dots, S_m\}$ 。
- 对每个句子 $S_i \in T$ ，进行分词和词性标注处理，并过滤掉停用词，只保留特定词性的词，如名词、动词、形容词。记 $S_i = \{t_{i,1}, t_{i,2}, \dots, t_{i,n}\}$ ，其中 $t_{i,n}$ 是保留后的**候选关键词**。
- 构建候选关键词图 $G=(V, E)$ ，其中 V 为节点集，由上步生成的候选关键词组成，然后采用共现关系构造任意两点之间的边，**两个节点之间存在边当且仅当候选关键词在长度为 k 的窗口中时共现**（ k 表示窗口大小，即最多共现 k 个词）。
- 根据TextRank关键词提取算法公式，**初始化权重，迭代地计算各个节点的权重**，直到收敛为止。
- 对节点的权重进行降序排序，得到最重要的前 n 个候选关键词。
- 查看这 n 个候选关键词是否有相邻出现的情况，相邻出现则组合成为多词关键词，否则单独成为关键词。



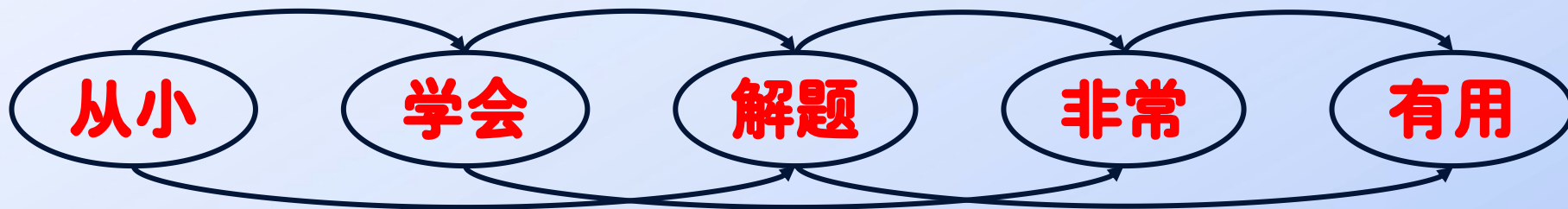
Jieba分词TextRank

- 步骤:

- 对每个句子进行分词和词性标注处理
- 过滤掉除指定词性外的其他单词，过滤掉出现在停用词表的单词，过滤掉长度小于2的词
- 将剩下的词中循环选择一个词，将其与其后面n个词分别组合成n条边。

- 例如:

- [他/从小/学会/了/解题/这/是/非常/有用/的]，假设窗口大小为3
- 每条边权值为1，当这条边在之后再次出现时，权值再在基础上加1.



边的权重可用在计算结点的出度和入度





LSA与LDA算法

- TF-IDF算法和TextRank算法各有所长，但在某些场合无法提取文本中的语义信息。
- 例如，一篇文章在讲各式各样的水果及其功效，当水果这一关键词没有直接出现在文本中时，两种算法都无法准确提取到水果这一关键词。主题模型可以有效的解决了这个问题。





主题模型

- **主题模型是在大量文档中发现潜在主题的一种统计模型。**直观上看，如果一篇文章有一个中心思想，那么一些特定词语会更频繁的出现。
- 例如，如果一篇文章是在讲汽车的，“汽车”和“驾驶”等词出现的频率会高些。一般情况下，一篇文章通常包含多种主题，而且每个主题所占比例各不相同。
- 因此，如果一篇文章10%的内容和“汽车”有关，90%的内容和“驾驶”有关，那么和“驾驶”相关的关键字出现的次数大概会是和“汽车”相关的关键字出现次数的9倍。主题模型试图用数学框架体现文档的这种特点。
- 主题模型认为文档是由主题组成的，而主题是词的一个概率分布，即每个词都是通过“文档以一定的概率选择某个主题，再从这个主题中以一定的概率选择某个词”这样一个过程得到的。





主题模型

- 主题模型能自动分析每个文档，统计文档内的词语，根据统计的信息断定当前文档含有哪些主题，以及每个主题所占的比例各为多少。
- 常见的主题模型主要有潜在语义分析（Latent Semantic Analysis, LSA）、概率潜在语义分析（Probabilistic Latent Semantic Analysis, PLSA）、潜在狄利克雷分布（Latent Dirichlet Allocation, LDA），以及基于深度学习的LDA2vec等。所有主题模型都基于以下两个相同的基本假设。
 - 每个文档包含多个主题。
 - 每个主题包含多个词。
- LSA和潜在语义索引（Latent Semantic Index, LSI）都是对文档的潜在语义进行分析，但是LSI在分析后，还会利用分析的结果建立相关的索引。二者通常被认为是同一种算法，只是应用的场景不同。





LSA算法-词语-文档矩阵

- LSA算法使用向量表示词和文档，词袋（Bag of Words, BOW）模型是文本向量化的最简单的模型。
- BOW模型就是将一个文本中的所有词语装进一个袋子里，不考虑其词法和语序的问题，每个词语都是独立的，将每一个单词都进行统计，同时计算每个单词出现的次数。
- BOW模型首先对文本进行分词，然后统计每个词在文档中出现的次数。如有两个短文本，通过分词后得到如下结果。
 - 张三/喜欢/外出/旅行，李四/也/喜欢/外出/旅行。
 - 张三/喜欢/看电影。





BOW (Bag Of Word, 词袋)

- 两个简单的文件:

- (1) John likes to watch movies. Mary likes movies too.
- (2) John also likes to watch football games.

- 基于以上两个文件，可以建构出下列单词清单:

- ["John", "likes", "to", "watch", "movies", "also", "football", "games", "Mary", "too"]
- (1) [1, 2, 1, 1, 2, 0, 0, 0, 1, 1]
- (2) [1, 1, 1, 1, 0, 1, 1, 1, 0, 0]





LSA算法

- 将这两个短文本所有词语装进一个袋子里，构成一个BOW。BOW中包含了七个不重复的词{张三、喜欢、外出、旅行、李四、也、看电影}。BOW模型规定每个文本向量长度为词袋中词的个数，文本向量的分量对应词袋中词出现的次数。
- 例如，在第一个短文本中，“喜欢”“外出”“旅行”3个词语在第一个短文本中出现了两次，“张三”“李四”“也”3个词语出现了一次，“看电影”一词则没有出现。
- 在BOW中对应的位置上标上词的出现次数，得到第一个短文本对应的向量为[1, 2, 2, 2, 1, 1, 0]。在第二个短文本中，“张三”“喜欢”“看电影”3个词各出现了一次，其他词语出现次数都为0，得到第二个短文本对应的向量为[1, 1, 0, 0, 0, 0, 1]。
- 因为BOW模型不考虑词语的顺序，所以得到的向量不会保存原始句子中词的顺序。





LSA算法

- 每个文本可以利用BOW模型得到一个向量，将所有文本的向量合在一起即可得到一个词语-文档矩阵。
- 例如，有词语-文档矩阵如下表所示，共有4个词和4个文档，词语-文档矩阵的每一列为文档（文本）的向量，每一行表示词在文档中出现的次数。

	文档1	文档2	文档3	文档4
词语1	0	1	1	2
词语2	1	1	0	0
词语3	1	2	0	2
词语4	0	1	1	1





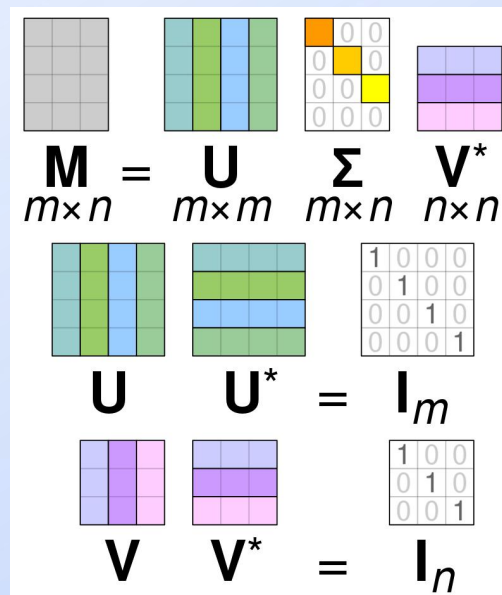
LSA算法-矩阵奇异值分解

- 如果在语料库中给出 m 个词和 n 个文档，可以构造一个 $m \times n$ 阶矩阵，其中每行代表一个词，每列代表一个文档。当拥有词语-文档矩阵 即可开始思考文本潜在的主题。但是词语-文档矩阵极有可能非常稀疏、噪声很大，并且在很多维度上非常冗余。
- 因此，为了找出能够捕捉词和文档关系的少数潜在主题，希望能降低矩阵 的维度。线性代数中的一种截断奇异值分解（Singular Value Decomposition, SVD）技术可以达到矩阵奇异值分解从而降维的目的。



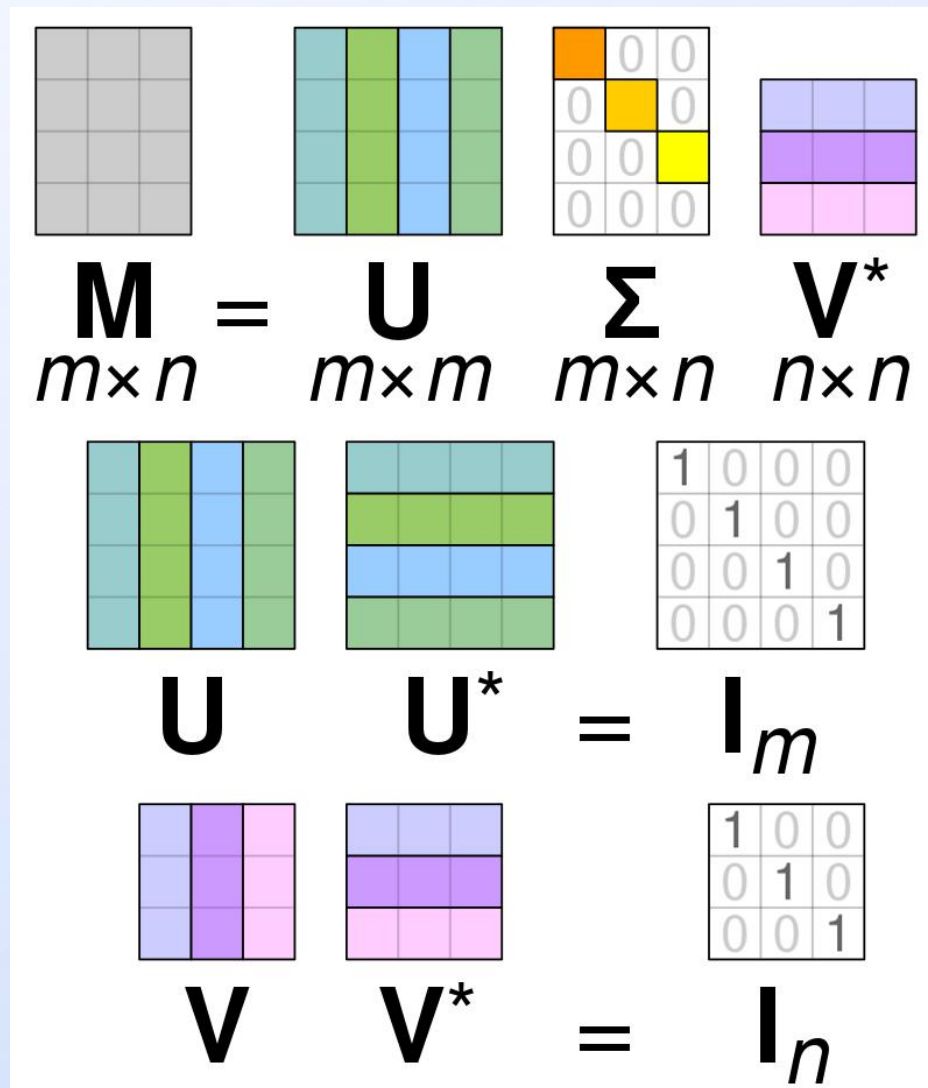
LSA算法

- 设矩阵 M 为一个 $m \times n$ 阶的矩阵，其中的元素全部属于实数域或复数域，那么矩阵 M 的SVD表示为 $M = U\Sigma V^T$ 。
- 其中， U 是一个 $m \times m$ 阶正交矩阵（行向量和列向量皆为正交的单位向量）、 Σ 是 $m \times n$ 阶半正定对角矩阵（对角元素为非负的对角阵），其主对角线上的每个元素称为奇异值， V^T 是 n 阶正交矩阵， U 的列向量称为左奇异向量， V^T 的列向量称右奇异向量，并且满足 $U^T U = I$ 和 $V^T V = I$ 。


$$\begin{array}{cccc} \begin{array}{|c|c|c|c|} \hline & & & \\ \hline \end{array} & \begin{array}{|c|c|c|c|} \hline & & & \\ \hline \end{array} & \begin{array}{|c|c|c|c|} \hline 0 & 0 & & \\ 0 & 0 & & \\ 0 & 0 & & \\ 0 & 0 & & \\ \hline \end{array} & \begin{array}{|c|c|c|c|} \hline & & & \\ \hline \end{array} \\ \mathbf{M} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^* \\ m \times n \quad m \times m \quad m \times n \quad n \times n \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline \end{array} & \begin{array}{|c|c|c|c|} \hline & & & \\ \hline \end{array} & \begin{array}{|c|c|c|c|} \hline 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ \hline \end{array} \\ \mathbf{U} \quad \mathbf{U}^* = \mathbf{I}_m \\ \begin{array}{|c|c|c|c|} \hline & & & \\ \hline \end{array} & \begin{array}{|c|c|c|c|} \hline & & & \\ \hline \end{array} & \begin{array}{|c|c|c|c|} \hline 1 & 0 & 0 & \\ 0 & 1 & 0 & \\ 0 & 0 & 1 & \\ 0 & 0 & 0 & 1 \\ \hline \end{array} \\ \mathbf{V} \quad \mathbf{V}^* = \mathbf{I}_n \end{array}$$



SVD (奇异值分解 Singular value decomposition)



The diagram illustrates the SVD decomposition of a matrix M into three matrices: U , Σ , and V^* .

Matrix M ($m \times n$): A 4x4 grid with a 2x2 block of non-zero elements (orange, yellow, yellow, yellow) in the top-left corner.

Matrix U ($m \times m$): A 4x4 grid with four columns of different colors (teal, green, blue, green).

Matrix Σ ($m \times n$): A 4x4 grid with a 2x2 block of non-zero elements (orange, yellow, yellow, yellow) in the top-left corner.

Matrix V^* ($n \times n$): A 4x4 grid with four rows of different colors (purple, purple, purple, pink).

The decomposition is shown as:

$$M = U \Sigma V^*$$

Matrix U ($m \times m$): A 4x4 grid with four columns of different colors (teal, green, blue, green).

Matrix U^* ($m \times m$): A 4x4 grid with four rows of different colors (teal, green, blue, green).

Matrix I_m ($m \times m$): A 4x4 grid with a 4x4 identity matrix (1s on the diagonal, 0s elsewhere).

The relationship is shown as:

$$U U^* = I_m$$

Matrix V ($n \times n$): A 4x4 grid with four columns of different colors (purple, purple, purple, pink).

Matrix V^* ($n \times n$): A 4x4 grid with four rows of different colors (purple, purple, purple, pink).

Matrix I_n ($n \times n$): A 4x4 grid with a 4x4 identity matrix (1s on the diagonal, 0s elsewhere).

The relationship is shown as:

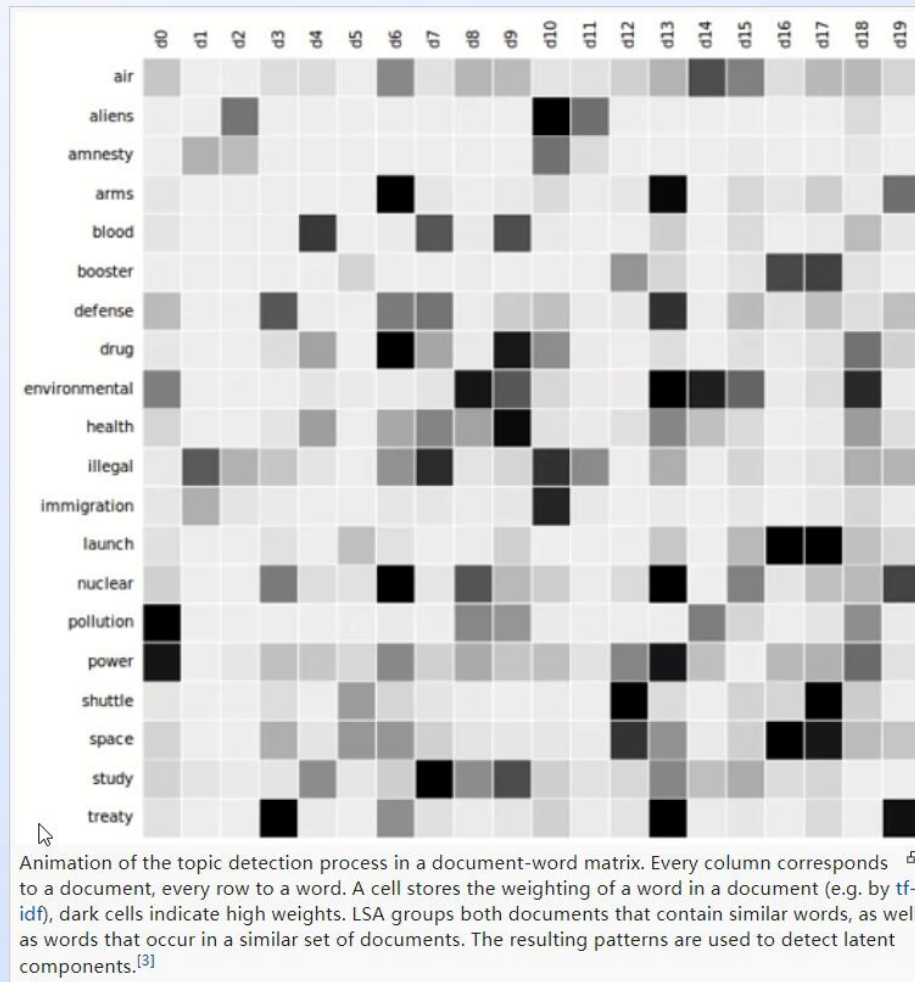
$$V V^* = I_n$$



LSA

- https://en.wikipedia.org/wiki/Latent_semantic_analysis

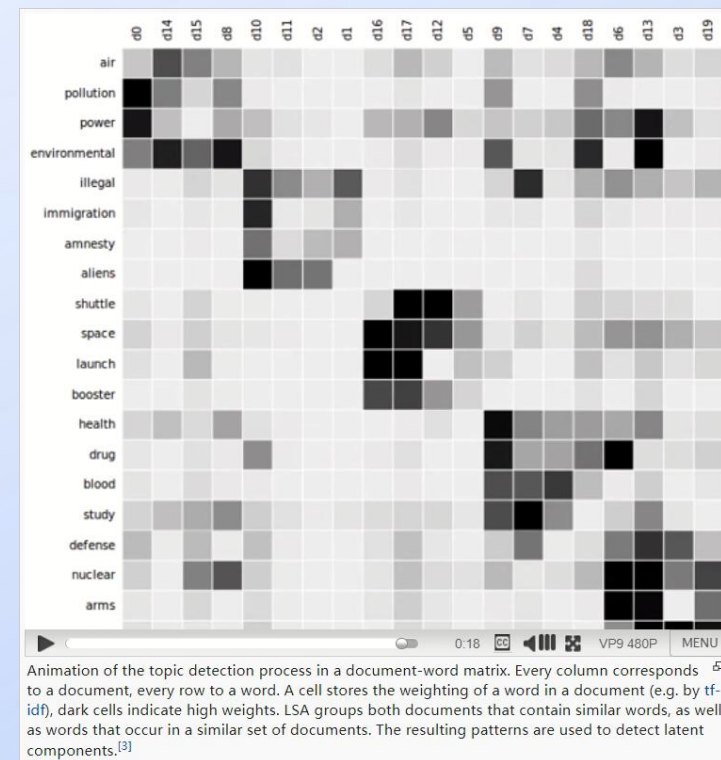
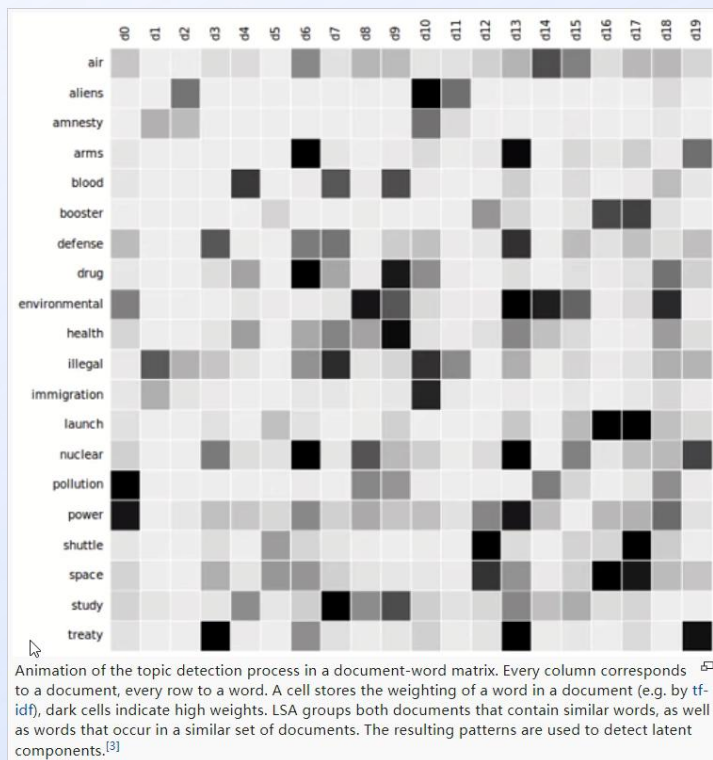
- 维基百科的解释



$$A = \pi r^2$$

LSA

- https://en.wikipedia.org/wiki/Latent_semantic_analysis
- 维基百科的解释



$$A = \pi r^2$$



LSA算法

- SVD有如下性质。
 - 一个 $m \times n$ 阶矩阵至多有 $p = \min(m, n)$ 个不同的奇异值，每个主题包含多个词。
 - 奇异值包含着矩阵中的重要信息，值越大表示越重要。
- 词语-文档矩阵A经过SVD后的U和 V^T 的维度较大，需要进行降维，原因有以下几点。
 - 原始的词语-文档矩阵维度太大并且过于稀疏，计算复杂度过高，无法准确地反映每个词是否出现在某些文档之中。
 - 矩阵降维可以对矩阵数据进行去噪，得到重要特征。
 - 矩阵降维可以降低同义词和多义词的影响，减少数据冗余。





LSA算法

- **截断奇异值分解**，就是奇异值从大到小排序，**取前r个非零奇异值对应的奇异向量代表矩阵A的主要特征**，如下式所示。

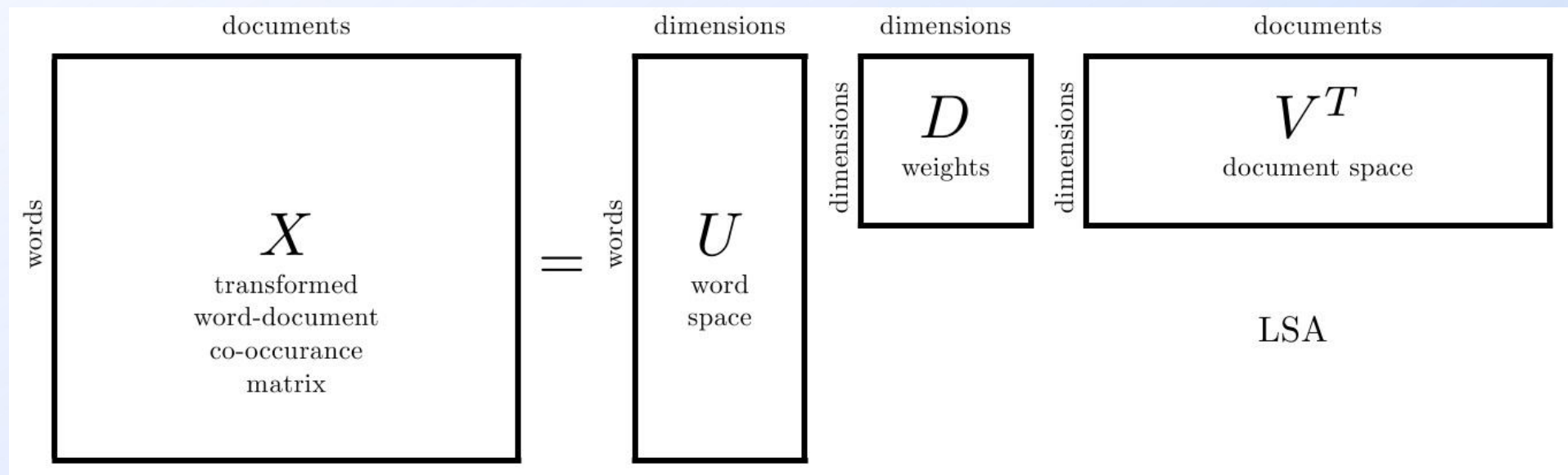
$$A_{m \times n} \approx U_{m \times r} \Sigma_{r \times r} V_{r \times n}^T$$

- 左奇异向量矩阵U代表词的部分特征，矩阵中的每一列由m个词按照一定的权重组合而来，它们相互独立并且各代表一个潜在语义，这r个潜在语义共同构成一个语义空间。
- 中间的奇异值矩阵Σ包含了词和文档的重要程度的信息，其数值越大，重要程度越高。
- 右奇异向量矩阵 V^T 则代表文档的部分特征。
- 通过SVD可以得到文档、词与主题、语义之间的相关性。通过这些文档向量和词向量，应用余弦相似度等度量评估不同文档的相似度、不同单词的相似度和词语与文档的相似度。



LSA (Latent Semantic Analysis)

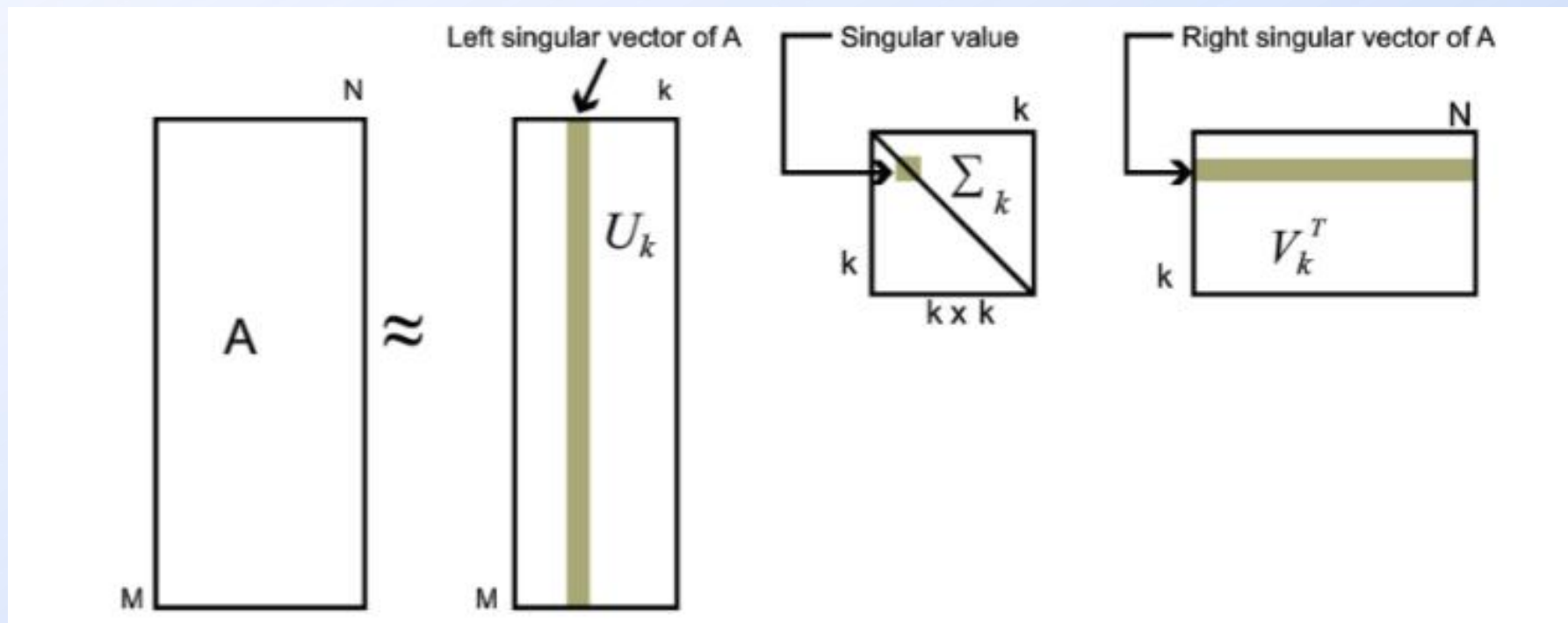
- LSA, 潜在语义分析
- LSA 将词和文档映射到潜在语义空间, 即矩阵D



大家暂不必关心D中各行各列的物理含义
通过SVD方法, 将X分解, 即将word和document通过D关联起来

SVD (Singular value decomposition)

- 在奇异值矩阵中按照从大到小排列，在很多情况下，前10%甚至1%的奇异值的和就占了全部的奇异值之和的99%以上的比例。也就是说，我们也可以用最大的 k 个的奇异值和对应的左右奇异向量来近似描述矩阵。即矩阵 A 只需要灰色的部分的小矩阵就可以近似描述了。



$$\frac{A}{r} = \pi r^2$$



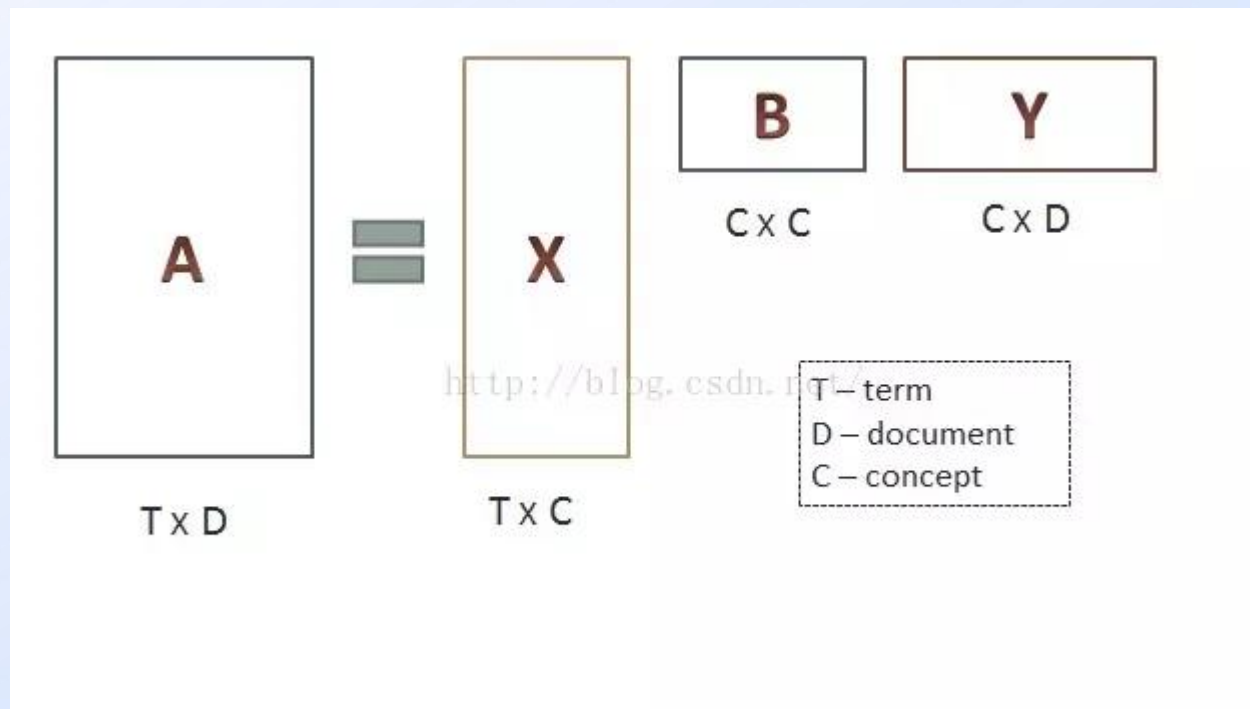
LSA算法步骤

- LSA关键词提取算法的具体步骤如下。
- 利用BOW模型将文档集中每个文档表示为向量。
- 将文档集中的所有词语和文档向量构成一个 $m \times n$ 阶的词语-文档矩阵。其中，文档集中的每一篇文档作为矩阵的列，文档集中的所有词语作为矩阵的行。
- 采用SVD，将该矩阵分解为3个矩阵，分别是 $m \times r$ 阶左奇异矩阵、 $r \times r$ 阶奇异值对角矩阵、 $r \times n$ 阶右奇异矩阵。
- 根据SVD的结果，取前 k 个非零奇异值对应的向量，用于代表该矩阵的主要特征，构建潜在语义空间，计算每个词语和每个文档之间的相似度，相似度最高的词将作为该文档的关键词。

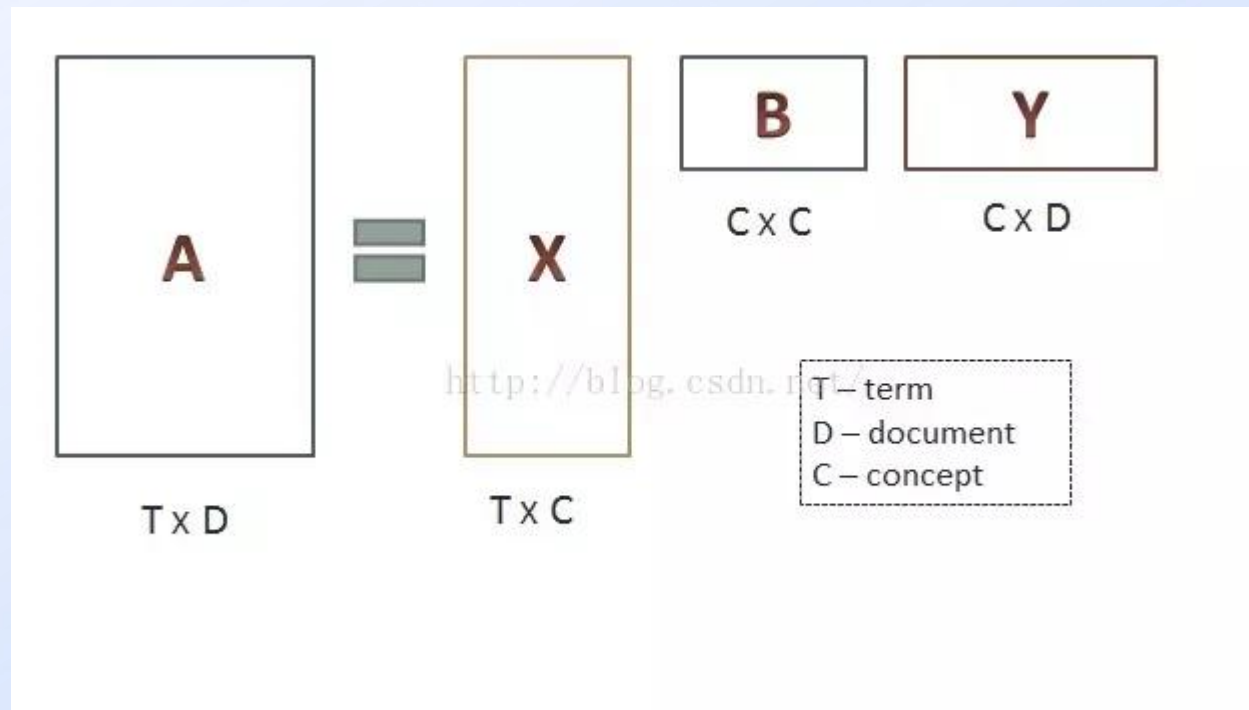


LSA

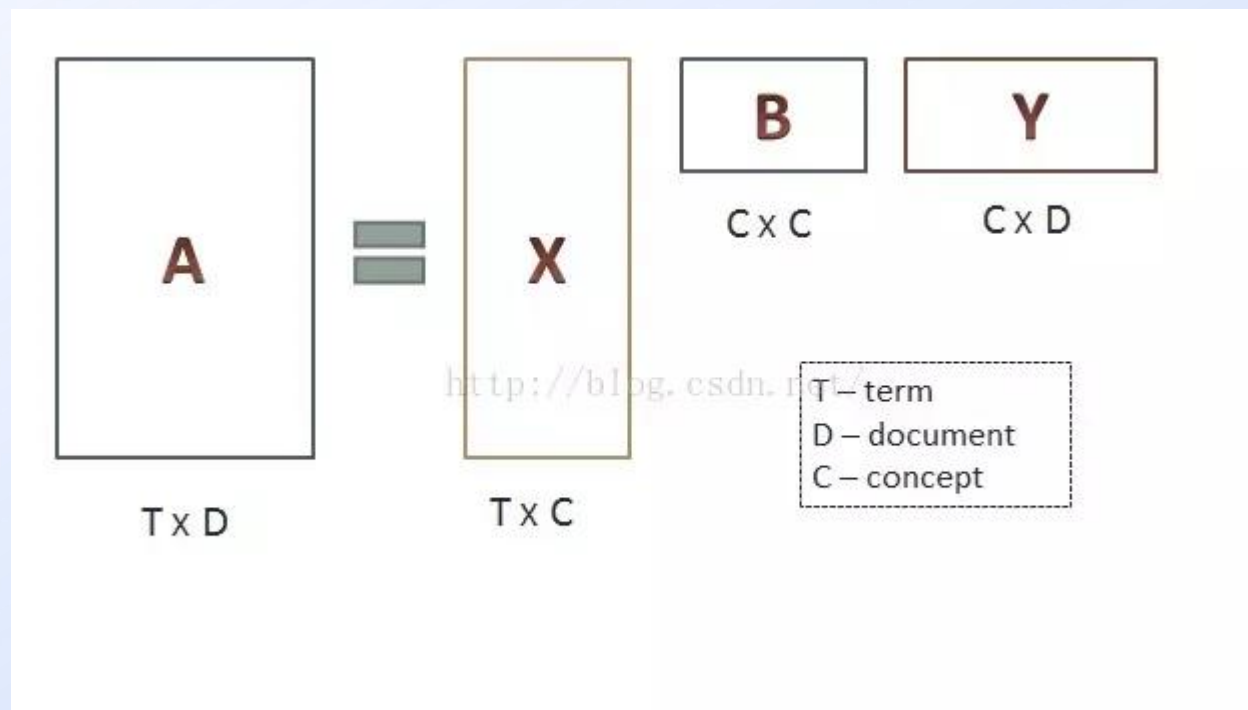
- (1) 词汇-文本矩阵A是一个稀疏矩阵，其行代表词语，其列代表文档。一般情况下，词-文档矩阵的元素是该词在文档中的出现次数BOW，也可以是该词语的tf-idf。



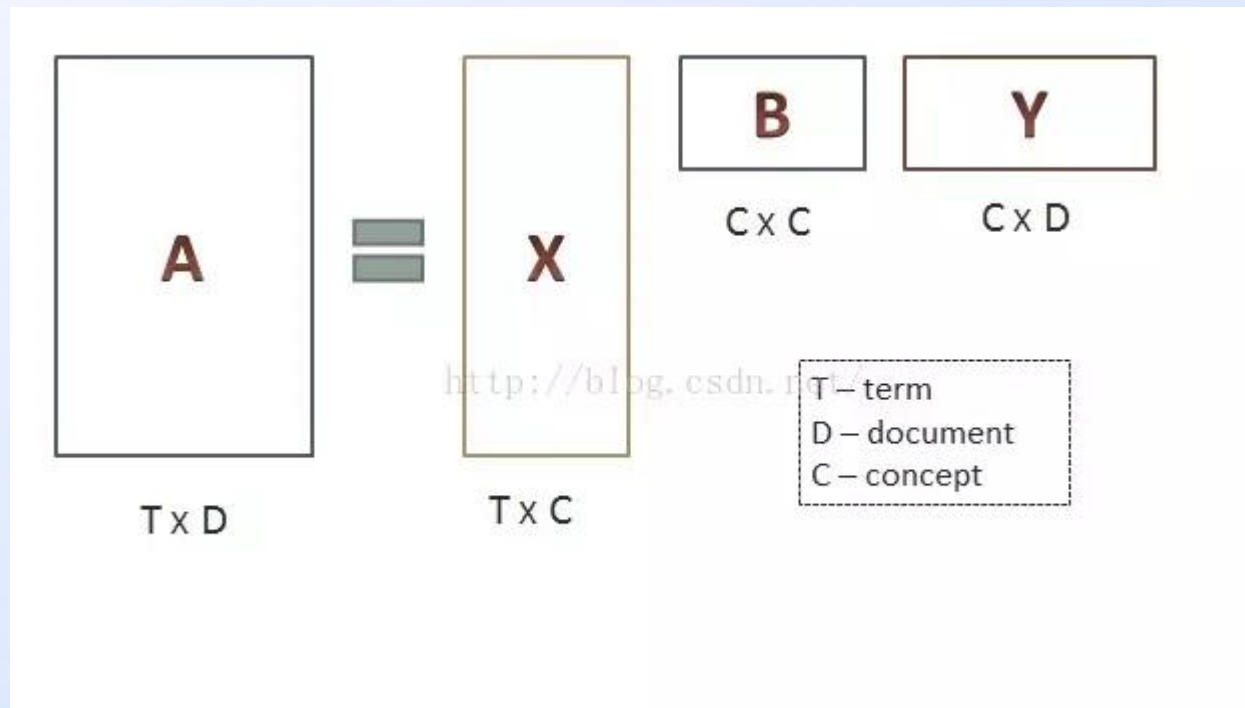
- (2) 小矩阵X是对词进行分类的一个结果，它的每一行表示一个词，每一列表示一个语义相近的词类，这一行中每个非零元素表示每个词在每个语义类中的重要性（或者说相关性）



- (3) 第二个小矩阵B表示词的类和文章的类之间的相关性



- (4) 矩阵Y是对文本进行分类的一个结果，它的每一行表示一个主题，每一列表示一个文档，这一列每个元素表示这篇文档在不同主题中的相关性





LSA

- 通过对大量的文本集进行统计分析，从中提取出词语的上下文使用含义。技术上通过SVD分解等处理，消除了同义词、多义词的影响，提高了后续处理的精度。

流程：

- (1) 分析文档集合，建立词汇-文本矩阵A.
- (2) 对词汇-文本矩阵进行奇异值分解.
- (3) 对SVD分解后的矩阵进行降维.
- (4) 使用降维后的矩阵构建潜在语义空间.





LSA

- 问题:

- SVD的物理意义似乎并不明确, 难以确切地表述矩阵特征值的缩减到底代表了什么含义?
- SVD的时间和空间复杂度很高


$$\frac{A}{r} = \pi r^2$$



基于LSA的关键词提取

- (1) BOW模型将文档表示为向量。
- (2) 构建词-文章矩阵A。
- (3) 对矩阵A进行SVD（奇异值分解）操作。
- (4) 对A进行降维，将词和文档都表示为K个主题构成的空间中的一个点，**计算每个词和文档的相似度，得到与每个文档中相似度最高的几个词为关键词。**





LSA算法

- 以下举例说明LSA关键词提取算法实现过程。
- 假设有词语-文档矩阵如下表所示，共有城市、密集型、必须、我国、行业、轨道交通、运营、里程这8个词，d1~d7分别代表了7个文档。矩阵中每一行代表词在对应文档中出现的次数，每一列表示每一个文档包含了哪些词。

词语	文档						
	d1	d2	d3	d4	d5	d6	d7
城市	0	0	1	1	0	0	1
密集型	0	0	1	1	0	0	0
必须	0	0	1	2	0	0	0
我国	1	0	0	0	0	0	1
行业	0	0	1	1	1	1	0
轨道交通	1	0	1	1	0	0	1
运营	0	1	0	0	1	0	0
里程	0	2	0	0	0	0	0





LSA算法

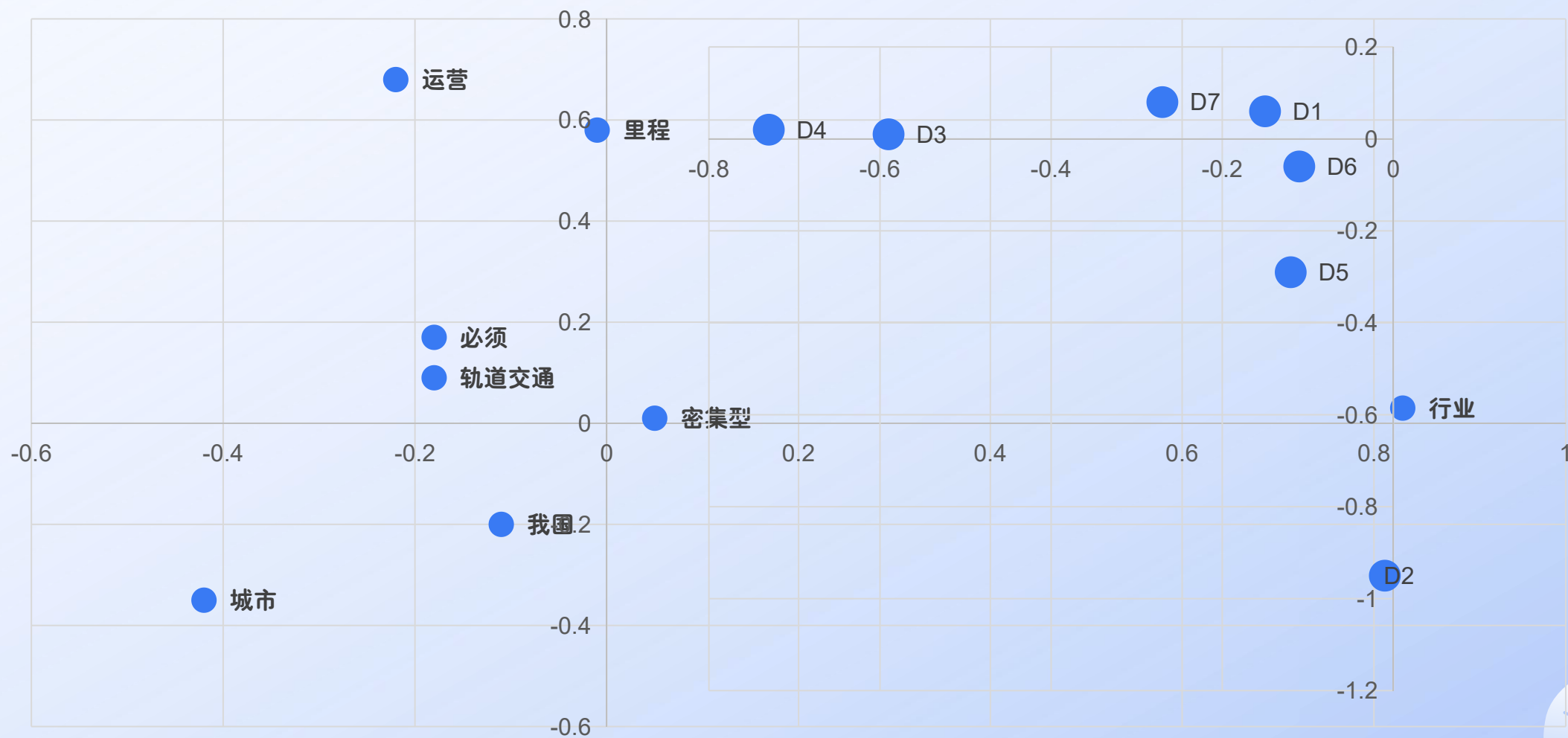
- 对词语-文档矩阵进行SVD，选取奇异值最大的3项，得到分解后的3个矩阵，从左到右分别是左奇异矩阵、奇异值矩阵和右奇异矩阵。

城市	-0.42	-0.35	-0.55
密集型	0.05	0.01	0.02
必须	-0.18	0.17	0.3
我国	-0.11	-0.2	-0.41
行业	0.83	0.03	-0.44
轨道交通	-0.18	0.09	-0.27
运营	-0.22	0.68	-0.42
里程	-0.01	0.58	0.01

3.75	0	0
0	2.3	0
0	0	1.94

d1	d2	d3	d4	d5	d6	d7
-0.15	-0.01	-0.59	-0.73	-0.12	-0.11	-0.27
0.06	-0.95	0.01	0.02	-0.29	-0.06	0.08
-0.59	-0.17	0.09	0.24	0.21	0.20	-0.69





$$A = \pi r^2$$



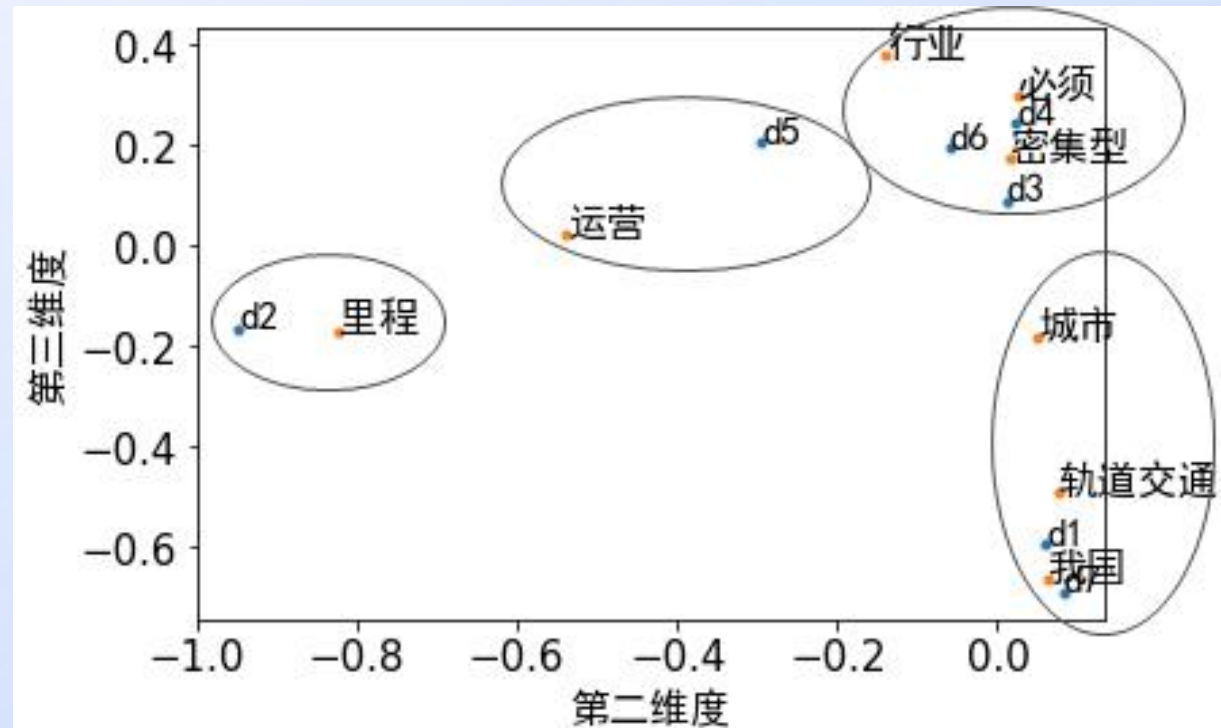
LSA算法

- 观察左奇异矩阵表和右奇异矩阵表可以发现，每个词和每个文档都可以用一个三维向量表示，如“城市”一词可用向量表示为 $(-0.42, -0.35, -0.55)$ ，文档d1可用向量表示为 $(-0.15, 0.06, -0.59)$ 。
- 这说明，通过SVD将每个词和每个文档都映射到一个三维空间上。可以将这个三维空间看作是一个三维语义空间，3个维度的潜在语义可以表示为如下形式。
 - 第一维度： $-0.42 \times \text{城市} + 0.05 \times \text{密集型} - 0.18 \times \text{必须} - 0.11 \times \text{我国} + 0.83 \times \text{行业} - 0.18 \times \text{轨道交通} - 0.22 \times \text{运营} - 0.01 \times \text{里程}$ 。
 - 第二维度： $-0.35 \times \text{城市} + 0.01 \times \text{密集型} + 0.17 \times \text{必须} - 0.2 \times \text{我国} + 0.03 \times \text{行业} + 0.09 \times \text{轨道交通} + 0.68 \times \text{运营} + 0.58 \times \text{里程}$ 。
 - 第三维度： $-0.55 \times \text{城市} + 0.02 \times \text{密集型} + 0.3 \times \text{必须} - 0.41 \times \text{我国} - 0.44 \times \text{行业} - 0.27 \times \text{轨道交通} - 0.42 \times \text{运营} + 0.01 \times \text{里程}$ 。



LSA算法

- 当词与文档同在一个空间上的时候，可以获得两个比较重要的信息。
 - 一是当两个词或两个文档在空间上的距离比较近的时候，可以认为两个词是近义词或两个文档有较高的相似度。
 - 二是当一个词与某个文档在空间上的距离比较近的时候，可以认为这个词是文档的关键词。
- 取左、右奇异向量矩阵的第二、第三个维度，投影到同一平面上，将各个词和文档标注在图中，距离相近的用椭圆线画出来，得到左、右奇异向量的空间投影图，如右图所示。





LSA算法

- 在左、右奇异向量空间投影图中，词与词之间或文档之间的距离相近，说明它们具有较高的相似度。文档d1和d7距离最近，相似度最高，因而被聚为一类。
- 同样地，文档d3、d4和d6也被聚为一类。而“城市”“轨道交通”“我国”3词距离很近，相似度高，也被聚为一类。
- 依照左、右奇异向量矩阵后两个维度的距离信息，可以近似地提取出文档中的近义词。另一方面，计算每个词语和每个文档之间的相似度，相似度较高的词可以认为是该文档的关键词。
- LSA算法利用SVD将词语、文档映射到更低维度的空间。低维空间去除了部分噪声，大大降低计算代价，也更容易发现同义词和相似的主题，更好地剖析词语和文档中的潜在语义。





LSA算法

- LSA模型仍存在许多不足之处:
- SVD计算复杂度高, 尤其对于文本处理, SVD用于高维度矩阵时效率较低。当新文档进入特征空间时, 需要重新训练模型。
- 没有解决多义词的问题, 每一个词语仅仅对应映射空间中的一个点, 多个含义的词语在映射空间中没有区分开。
- LSA受BOW模型的影响, 会忽略文档中句子的先后顺序。
- LSA缺乏严谨的统计基础, 难以直观进行解释。





LDA算法

- 由于LSA算法存在诸多不足，于是产生了PLSA算法，它是一个概率模型算法，采用更为符合文本特性的多项式分布和最大期望算法（Expectation-Maximization algorithm, EM）算法拟合概率分布信息，使得模型中的变量的概率分布都有更好的解释。
- 但PLSA算法仍然不够完善，它只能生成所在文档集的文档的模型，当使用EM算法进行反复迭代时，计算量会很大，当文档和词语数量增多时，模型训练参数的值也会随之线性增加，容易导致过度拟合。因此，在对PLSA算法修改的基础上，LDA算法被提出。





- LDA模型认为主题可以由一个词汇分布来表示，而文章可以由主题分布来表示。
- 比如有两个主题，美食和美妆。LDA的两个主题可以由词汇分布表示，他们分别是：
 - {面包：0.4，火锅：0.5，眉笔：0.03，腮红：0.07}
 - {眉笔：0.4，腮红：0.5，面包：0.03，火锅：0.07}
- 对于两篇文章，LDA认为文章可以由主题分布这么表示：
 - 《美妆日记》{美妆：0.8，美食：0.1，其他：0.1}
 - 《美食探索》{美食：0.8，美妆：0.1，其他：0.1}





LDA算法-LDA模型

- LDA模型是应用比较广泛的一种主题模型，包含词、主题和文档3层结构。LDA模型假定词语之间没有顺序，所有的词语都无序地放在一个袋子里，并且认为一个文档可以有多个主题，每个主题对应有不同的词语。
- 假设语料库包含 m 个词， n 个文档，每个文档包含 k 个主题。其中 w_i 、 d_j 分别表示词和文档， t_k 代表第 k 个主题。LDA模型可以表示为如下式所示。

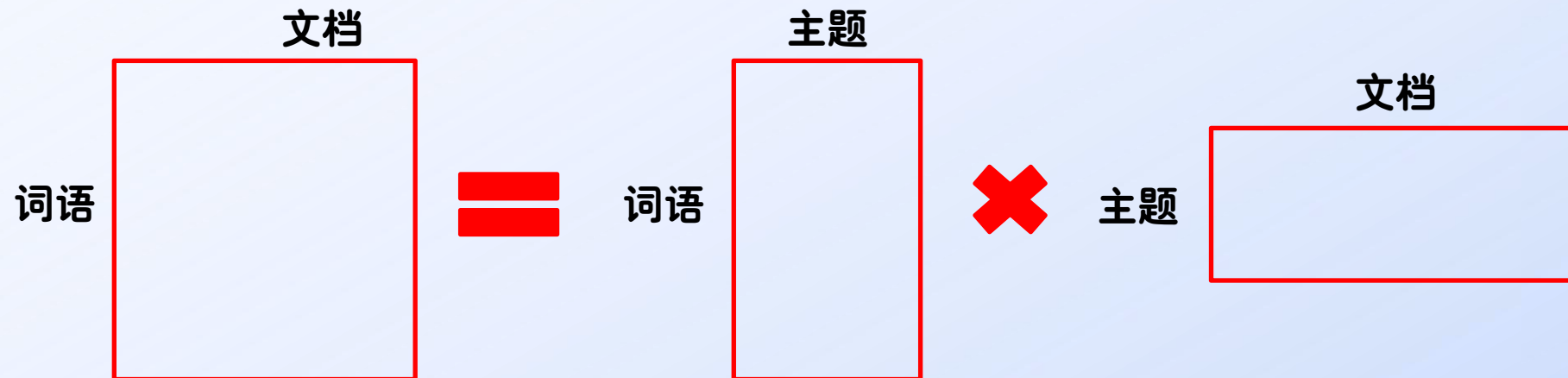
$$P(w_i | d_j) = \sum_{k=1}^K P(w_i | t_k) \times P(t_k | d_j)$$

$$P(\text{词语} | \text{文档}) = \sum P(\text{词语} | \text{主题}) \times P(\text{主题} | \text{文档})$$





- 这个概率可以表示为矩阵：



其中，“文档-词语”矩阵表示每个文档中每个单词的词频，即出现的概率；
“主题-词语”矩阵表示每个主题中每个单词的出现概率；
“文档-主题”矩阵表示每个文档中每个主题出现的概率。





LDA算法

- 假设 $P(w|d)$ 、 $P(w|t)$ 和 $P(t|d)$ 分别称为词-文档、词-主题和主题-文档概率分布矩阵，则上个式子中3个概率分布矩阵分别表示如下，LDA主题模型的矩阵形式可以表示为 $P(w|d)$ 。
- 当LDA模型对一篇文档进行关键词抽取时，根据LDA模型能够得到每个主题，生成每个词的概率，然后将每个主题中的概率最大的前 k 个词取出并作为该文档的关键词。

$$P(w|d) = \begin{bmatrix} P(w_1|d_1) P(w_1|d_2) \cdots P(w_1|d_n) \\ P(w_2|d_1) P(w_2|d_2) \cdots P(w_2|d_n) \\ \cdots & \cdots & \cdots \\ P(w_m|d_1) P(w_m|d_2) \cdots P(w_m|d_n) \end{bmatrix}$$

$$P(w|t) = \begin{bmatrix} P(w_1|t_1) P(w_1|t_2) \cdots P(w_1|t_K) \\ P(w_2|d_1) P(w_2|t_2) \cdots P(w_2|t_K) \\ \cdots & \cdots & \cdots \\ P(w_m|t_1) P(w_m|t_2) \cdots P(w_m|t_K) \end{bmatrix}$$

$$P(t|d) = \begin{bmatrix} P(t_1|d_1) P(t_1|d_2) \cdots P(t_1|d_n) \\ P(t_2|d_1) P(t_2|d_2) \cdots P(t_2|d_n) \\ \cdots & \cdots & \cdots \\ P(t_K|d_1) P(t_K|d_2) \cdots P(t_K|d_n) \end{bmatrix}$$



LDA算法-LDA模型的概率分布

- LDA模型的概率分布可以描述为已知 $P(w|d)$ 概率分布矩阵，求概率分布矩阵 $P(w|t)$ 和 $P(t|d)$ 。模型的主题的先验分布和主题中词的先验分布假设如下。
- 假设文档主题的先验分布为Dirichlet分布，即对于任一文档 i ，其主题分布如下式所示，其中， θ 为文档主题的超参数， α 是一个 K 维向量。

$$\vec{\theta}_i \sim \text{Dir}(\vec{\alpha}), \quad i = 1, 2, \dots, n$$

- 假设主题中词的先验分布为Dirichlet分布，即对第 k 个主题，其词分布如下式所示。其中 φ 为分布的超参数， β 是一个 m 维向量。

$$\vec{\varphi}_k \sim \text{Dir}(\vec{\beta}), \quad k = 1, 2, \dots, K$$





LDA模型生成文档的步骤

- LDA主题模型是一种文档生成模型，它认为一篇文档有多个主题，每个主题对应着不同的词。
- 一篇文档的生成过程，首先是以一定的概率选择某个主题，然后再在这个主题下以一定的概率选出某一个词，这样就生成了这篇文档的第一个词。不断重复这个过程，就生成了整篇文章。
- 这里假定词与词之间是没有顺序的，即所有词无序的堆放在一个大袋子中，称之为词袋，这种方式使得算法相对简化一些。在LDA模型中，一篇文档生成的步骤如下。
 - 按照先验概率 选择一篇文档 d_i 。
 - 人为设置超参数，获得主题分布 $\vec{\theta}_i$ 。



目录 CONTENTS



01

关键词提取
技术简介

02

关键词提取
算法

03

自动提取文
本关键词



自动提取文本关键词

- 本节根据算法原理自定义TF-IDF、TextRank和LSA三种算法的函数，并通过实例完成关键词自动提取。
- 关键词提取流程主要包括数据预处理、算法实现和结果分析等步骤。
- 在提取关键词之前，需要先输入准备好文档。





文本预处理

- 文档准备好之后，需要加载停用词，并对当前文档进行分词和词性标注，过滤一些对提取关键词帮助不大的词性。
- 将名词作为候选关键词，在过滤词性中只留下名词，并且删除长度小于或等于1的无意义词语。文本预处理的步骤如下。
 - 加载停用词文件stopword.txt并按行读取文件中的停用词，将文本中多余的换行符进行替换，最终获取停用词列表。其中自定义Stop_words函数用于获取停用词列表。
 - 对当前文档过滤停用词。自定义Filter_word函数用于对当前文档进行处理，输入参数为当前文档内容。处理后的文档存放在filter_word变量中，它是一个包含着多个字符串的列表。
 - 对文档集corpus.txt过滤停用词。文档集选取国内2012年6-7月期间，搜狐新闻中国际、体育、社会、娱乐等18个频道的新闻内容，其中包含多行文本内容，读取时以列表的形式追加，每个文档以字符串的形式存放在列表当中。
 - 自定义Filter_words函数用于对文档集进行处理，输入参数是文档集路径。处理后的文档集存放在document变量中，它是一个包含着多个列表的列表，相当于将多个filter_word变量组合为一个列表。

$$A = \pi r^2$$



TF-IDF算法

- 自定义的TF-IDF算法函数名为`tf_idf`，其算法实现包括以下3个步骤。
- 对TF值进行统计。调用自定义`Filter_word`函数处理当前文档，统计当前文档中每个词的TF值。
- 对IDF值进行统计。调用自定义`Filter_words`函数处理文档集，统计IDF值。
- 对TF值和IDF值进行统计，二者结果相乘，得到TF-IDF值。





TextRank算法

- 自定义TextRank算法的函数名为TextRank，其算法实现包括以下3个步骤。
- 构建每个节点对应的窗口集合，当不同窗口中出现相同的词语时，相互连接形成边。
- 构建以边相连的关系矩阵，对矩阵进行归一化。
- 根据TextRank算法公式计算对应的TextRank值，提取关键词。





LSI算法

- 由于gensim库中只定义了LSI模型，而LSA算法和LSI算法原理基本一致，因此这里用LSI模型替代LSA模型。自定义LSI算法的函数名为lsi，其算法实现包括以下3个步骤。
- 构建基于文档集的词空间。使用BOW模型对每篇文档进行向量化，得到每一篇文档对应的稀疏向量。向量中包括向量的id和词频，其中id表示文档中每一个词语的索引，词频表示词语出现在文档中的次数，都以数字的形式表示。
- 构建TF-IDF模型，在此基础上加入向量化后的文档集语料corpus，结合成为经过TF-IDF加权的文档向量。基于SVD建立主题模型，得到当前文档和主题之间的分布。
- 采用余弦相似度计算相似度，求得当前文档与文档中的词语的相似度，相似度最高的前十个词作为当前文档的关键词。





LSI算法

- 基于当前文档，在3种模型中，TF-IDF模型的结果较好，其次是TextRank算法，最后是LSI模型。LSI模型是较早期的主题模型，存在诸多不足之处，因此用于提取关键词时效果不太理想，相较之下TF-IDF算法和TextRank算法结果会比较好。
- 由于提取关键词时，有些词语在文档中出现的频次、词性等信息比较接近，因此提取关键词时出现的概率很有可能一样，多次运行程序时，关键词出现顺序会往前或往后改变。





Python下的关键词提取

- 第三方库gensim

- Gensim (generate similarity) 是一个简单高效的自然语言处理Python库，用于抽取文档的语义主题 (semantic topics) 。
- Gensim的输入是原始的、无结构的数字文本 (纯文本)，内置的算法包括Word2Vec, FastText, 潜在语义分析 (Latent Semantic Analysis, LSA), 潜在狄利克雷分布 (Latent Dirichlet Allocation, LDA) 等，通过计算训练语料中的统计共现模式自动发现文档的语义结构。
- 这些算法都是非监督的，这意味着不需要人工输入——仅仅需要一组纯文本语料。一旦发现这些统计模式后，任何纯文本 (句子、短语、单词) 就能采用语义表示简洁地表达。





准备预料和文档向量化

- `import gensim`
- `from gensim import corpora`
- `document = Filter_words('data_c6/small_corpus.txt')`
 - #读取语料文件，并分词和去除停用词，得到document.
- `print("document:",document)`
- `dictionary = corpora.Dictionary(document)` # 生成基于文档集的语料
- `print("dictionary:",dictionary.token2id)`
- `corpus = [dictionary.doc2bow(doc) for doc in document]` # 文档向量化
- `print("corpus:",corpus)`

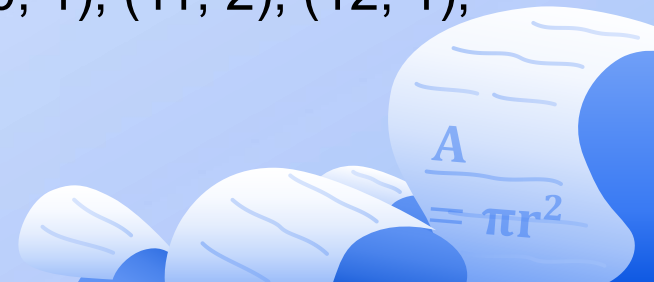




春娃娃真调皮。春风次他他就去门。春娃娃跑到布谷息家里，轻轻对布谷鸟吹了一口气，说：“布谷鸟姐姐，快起床吧！”布谷，布谷！”布谷鸟清了清嗓子唱起了歌。春娃娃一会儿跑到柳对姐那里，从早到晚不停地给柳风姐姐梳头发。柳树明姐的头发由短变长，可漂亮了。春娃娃一会儿又去催桃花姐姐开花，桃花姐姐绽开了粉红的笑脸。春娃娃跑累了，坐在山坡上，看着一群小朋友在草地上，一边玩一边唱：“春天来，春天来，柳树绿，桃花开，布谷鸟儿唱起歌，春娃娃呀多可爱。



```
document: [['娃娃', '调皮', '春风', '娃娃', '布谷', '布谷鸟', '布谷鸟', '姐姐', '布谷', '布谷', '布谷鸟', '嗓子', '娃娃', '柳风', '姐姐', '柳树', '明姐', '头发', '娃娃', '桃花', '姐姐', '开花', '桃花', '姐姐', '笑脸'], ['娃娃', '山坡', '小朋友', '草地', '柳树', '桃花开', '布谷鸟', '娃娃']]
dictionary: {'嗓子': 0, '头发': 1, '姐姐': 2, '娃娃': 3, '布谷': 4, '布谷鸟': 5, '开花': 6, '明姐': 7, '春风': 8, '柳树': 9, '柳风': 10, '桃花': 11, '笑脸': 12, '调皮': 13, '小朋友': 14, '山坡': 15, '桃花开': 16, '草地': 17}
corpus: [[(0, 1), (1, 1), (2, 4), (3, 4), (4, 3), (5, 3), (6, 1), (7, 1), (8, 1), (9, 1), (10, 1), (11, 2), (12, 1), (13, 1)], [(3, 2), (5, 1), (9, 1), (14, 1), (15, 1), (16, 1), (17, 1)]]
```

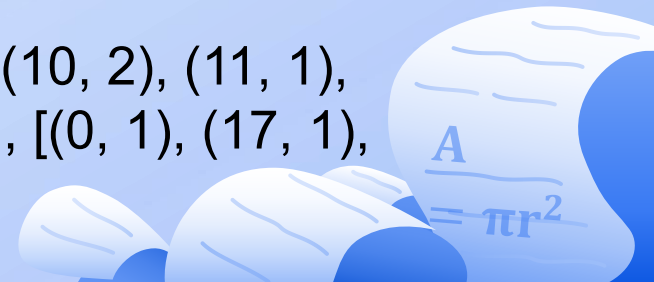




小乌龟兄妹俩想开一家小店，开什么店好呢？他们去问大象。大象说：“我开花店，可以用长鼻子给鲜花喷水。”他们去问河马。河马说：“我开气球店，可以用大嘴巴吹出最大的泡泡！”袋鼠妈妈在开什么店呢？袋鼠妈妈告诉他们：“我开书报店，把书报装到大口袋里，带上很多很多的书和报纸，走到哪里，卖到哪甲小乌龟兄妹很难过：“我们没有大鼻子，没有大嘴巴，也没有大口袋，怎么办呢？”小乌龟兄妹很伤心的在森林里走着。在路上，他们发现很多动物都因为找不到工作而感到难过。于是，他们开了一家烧饼店，嘿，没想到生意还挺不错



```
document: [['乌龟', '兄妹俩', '小店'], ['大象', '大象', '开花', '鼻子', '鲜花'], ['河马', '河马', '气球', '大嘴巴', '泡泡', '袋鼠', '妈妈', '袋鼠', '妈妈', '书报', '书报', '口袋', '报纸'], [], ['乌龟', '兄妹', '大鼻子', '大嘴巴', '口袋'], ['乌龟', '兄妹', '伤心', '森林', '动物', '烧饼']]
dictionary: {'乌龟': 0, '兄妹俩': 1, '小店': 2, '大象': 3, '开花': 4, '鲜花': 5, '鼻子': 6, '书报': 7, '口袋': 8, '大嘴巴': 9, '妈妈': 10, '报纸': 11, '气球': 12, '河马': 13, '泡泡': 14, '袋鼠': 15, '兄妹': 17, '大鼻子': 18, '伤心': 19, '动物': 20, '森林': 21, '烧饼': 22}
corpus: [[(0, 1), (1, 1), (2, 1)], [(3, 2), (4, 1), (5, 1), (6, 1)], [(7, 2), (8, 1), (9, 1), (10, 2), (11, 1), (12, 1), (13, 2), (14, 1), (15, 1), (16, 1)], [], [(0, 1), (8, 1), (9, 1), (17, 1), (18, 1)], [(0, 1), (17, 1), (19, 1), (20, 1), (21, 1), (22, 1)]]
```





gensim的TFIDF向量化

- `tf_idf_model = models.TfidfModel(corpus)`
- `vec = [(0,1),(4,1)]`
- `print(tf_idf_model[vec])`

语料

document: [['娃娃', '调皮', '春风', '娃娃', '布谷', '布谷鸟', '布谷鸟', '姐姐', '布谷', '布谷', '布谷鸟', '嗓子', '娃娃', '柳风', '姐姐', '柳树', '明姐', '头发', '娃娃', '桃花', '姐姐', '开花', '桃花', '姐姐', '笑脸'], ['娃娃', '山坡', '小朋友', '草地', '柳树', '桃花开', '布谷鸟', '娃娃']]

dictionary: {'嗓子': 0, '头发': 1, '姐姐': 2, '娃娃': 3, '布谷': 4, '布谷鸟': 5, '开花': 6, '明姐': 7, '春风': 8, '柳树': 9, '柳风': 10, '桃花': 11, '笑脸': 12, '调皮': 13, '小朋友': 14, '山坡': 15, '桃花开': 16, '草地': 17}

corpus: [[(0, 1), (1, 1), (2, 4), (3, 4), (4, 3), (5, 3), (6, 1), (7, 1), (8, 1), (9, 1), (10, 1), (11, 2), (12, 1), (13, 1)], [(3, 2), (5, 1), (9, 1), (14, 1), (15, 1), (16, 1), (17, 1)]]

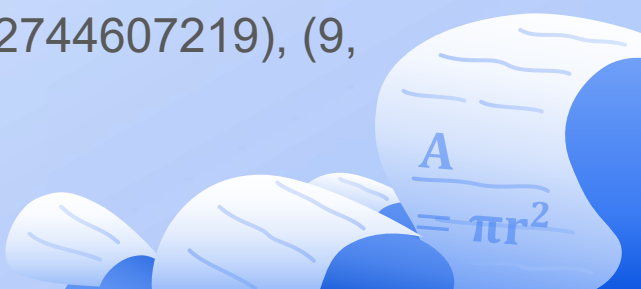
结果输出: [(0, 0.7071067811865475), (4, 0.7071067811865475)]





LSI

- from gensim import **models**
- lsi = **models**.LsiModel(corpus, id2word=dictionary, num_topics=10)
- print(lsi)
 - # 构建lsiLSI模型，函数包括3个参数：文档向量、文档集语料id2word和主题数目num_topics，
 - #id2word可以将文档向量中的id转化为文字
 - 输出：LsiModel<num_terms=18, num_topics=4, decay=1.0, chunksize=20000>
- vec vec = [(0,1),(4,1),(6,1)] #词袋向量
- print(lsi[tfidf[vec]])#tfidf向量
 - 输出：[(0, 0.0008953875360387506), (1, 0.0012371352663088013), (2, 0.0002520607712190043), (3, 0.004195556575812354), (4, -0.0004640787290279746), (5, 0.0012259520506923245), (6, -0.0005809912382290797), (7, -0.00011880290550219186), (8, -0.0004749802744607219), (9, 0.0002797073729715784)]


$$\frac{A}{r} = \pi r^2$$



LSI关键词提取

- 计算语料库中文档的lsi向量和待处理当前文档的lsi向量
- `word_topic = lsi[word_corpus]` #语料库
- `text_topic = lsi[text_corpus]` #待处理语料
- 计算当前文档和每个词语的主题分布相似度，例如余弦相似度，相似度越高越能代表这篇：

两个向量间的余弦值可以通过使用欧几里得点积公式求出：

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta.$$

给定两个属性向量， A 和 B ，其余弦相似性 θ 由点积和向量长度给出，如下所示：

$$\text{similarity} = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i \times B_i}{\sqrt{\sum_{i=1}^n (A_i)^2} \times \sqrt{\sum_{i=1}^n (B_i)^2}}.$$

$$A = \pi r^2$$



LDA

- `from gensim.models.ldamodel import LdaModel`
- `ldamodel = LdaModel(corpus, num_topics=5, id2word = dictionary, passes=30)`
 - `#num_topics`: 必须。LDA 模型要求用户决定应该生成多少个主题。
 - `#id2word`: 必须。LdaModel 类要求我们之前的 dictionary 把 id 都映射成为字符串。
 - `#passes`: 可选。模型遍历语料库的次数。遍历的次数越多，模型越精确。但是对于非常大的语料库，遍历太多次会花费很长的时间。





- `print("print_topic:",ldamodel.print_topics(num_topics=5, num_words=15))`
- `print_topic: [(0, '0.056**"娃娃" + 0.056**"姐姐" + 0.056**"布谷鸟" + 0.056**"布谷" + 0.056**"桃花" + 0.056**"柳树" + 0.056**"笑脸" + 0.056**"调皮" + 0.056**"柳风" + 0.056**"嗓子" + 0.056**"春风" + 0.056**"头发" + 0.056**"明姐" + 0.056**"开花" + 0.056**"草地")`, (1, '0.056**"柳树" + 0.056**"布谷鸟" + 0.056**"娃娃" + 0.056**"草地" + 0.056**"小朋友" + 0.056**"桃花开" + 0.056**"山坡" + 0.056**"调皮" + 0.056**"笑脸" + 0.056**"嗓子" + 0.056**"柳风" + 0.056**"头发" + 0.056**"开花" + 0.056**"明姐" + 0.056**"春风"), (2, '0.190**"娃娃" + 0.103**"柳树" + 0.103**"草地" + 0.103**"桃花开" + 0.103**"山坡" + 0.103**"小朋友" + 0.103**"布谷鸟" + 0.017**"调皮" + 0.017**"笑脸" + 0.017**"嗓子" + 0.017**"柳风" + 0.017**"头发" + 0.017**"开花" + 0.017**"春风" + 0.017**"明姐"), (3, '0.056**"柳树" + 0.056**"布谷鸟" + 0.056**"娃娃" + 0.056**"草地" + 0.056**"桃花开" + 0.056**"山坡" + 0.056**"小朋友" + 0.056**"调皮" + 0.056**"笑脸" + 0.056**"嗓子" + 0.056**"柳风" + 0.056**"头发" + 0.056**"春风" + 0.056**"明姐" + 0.056**"开花"), (4, '0.147**"娃娃" + 0.147**"姐姐" + 0.112**"布谷鸟" + 0.112**"布谷" + 0.077**"桃花" + 0.042**"开花" + 0.042**"明姐" + 0.042**"头发" + 0.042**"春风" + 0.042**"嗓子" + 0.042**"柳风" + 0.042**"调皮" + 0.042**"笑脸" + 0.042**"柳树" + 0.007**"草地"))]



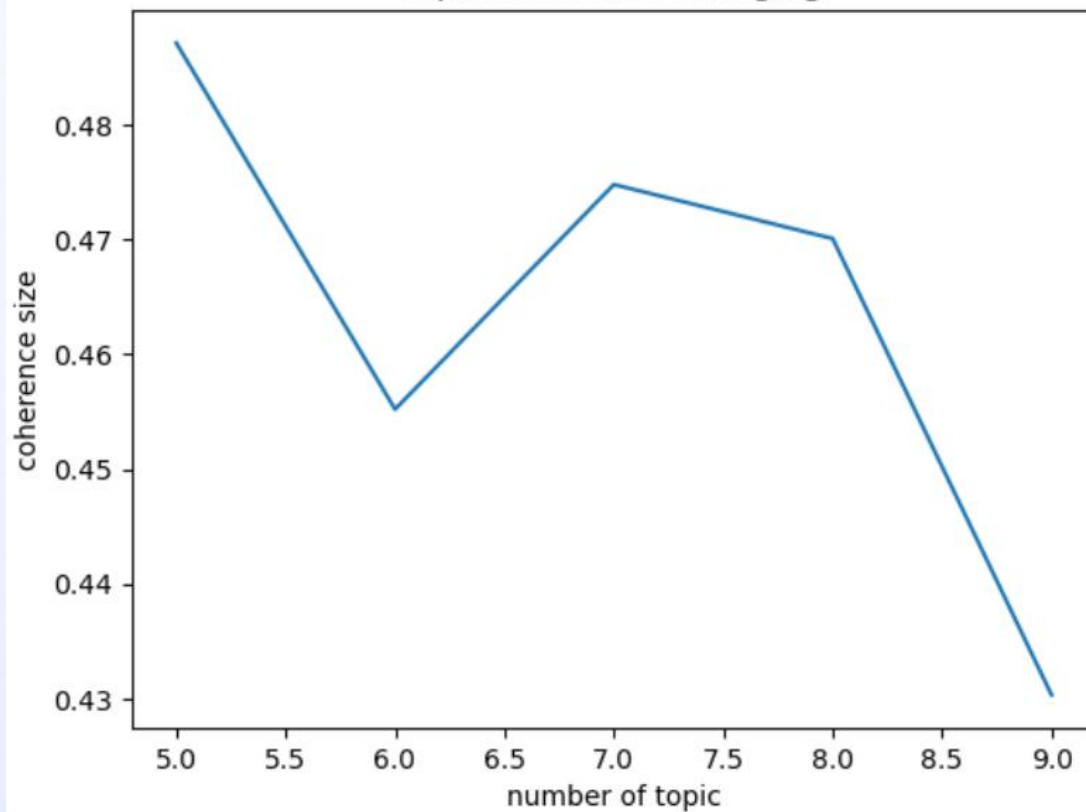


- `print("log_perplexity:",ldamodel.log_perplexity(corpus))`
 - 输出: `log_perplexity: -3.484183835260796`
- `ldacm = CoherenceModel(model=ldamodel, texts=document, dictionary=dictionary, coherence='c_v')`
- `print("get_coherence:",ldacm.get_coherence())`
 - 输出: `get_coherence: 0.41364573741491173`
- LDA主题模型的指标有困惑度 (perplexity) 和主题一致性 (coherence) , 困惑度越低或者一致性越高说明模型越好。

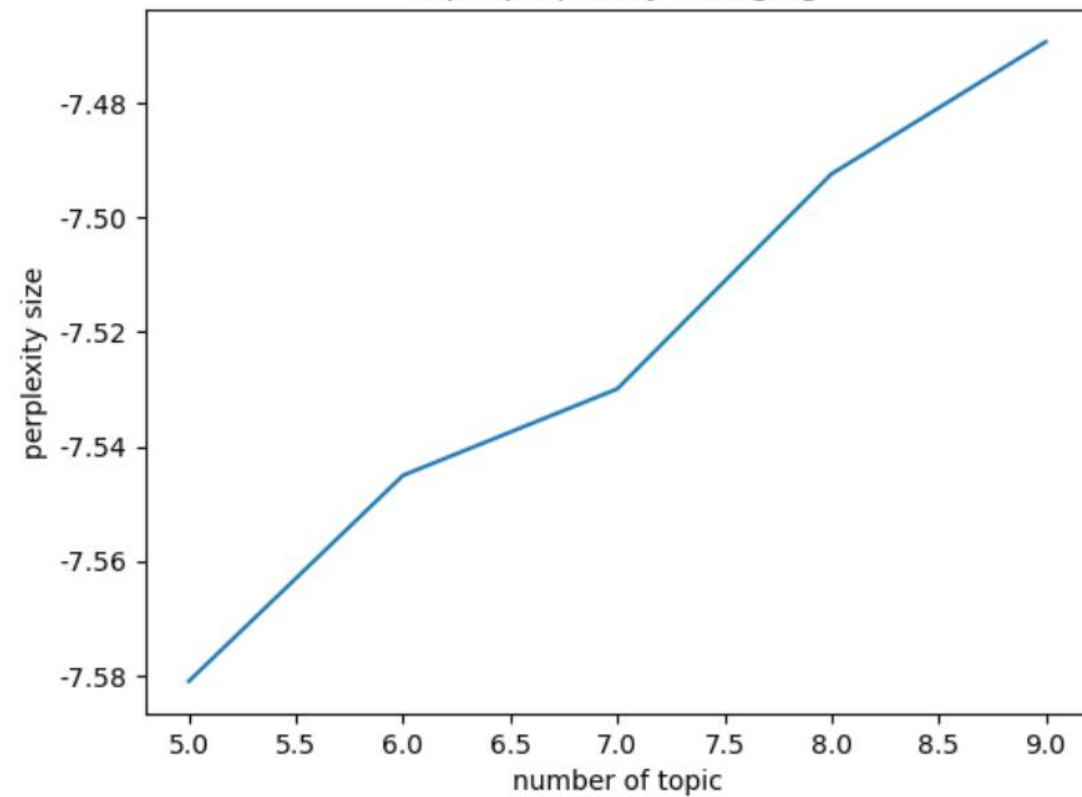




topic-coherence changing



topic-perplexity changing



$$A = \pi r^2$$



小结

- 本章主要介绍了关键词提取技术的3种算法。
- 首先对关键词提取技术做了简单介绍。其次对TF-IDF算法的基本原理进行阐述，并简单举例说明。
- 接着通过引入PageRank算法，给出了TextRank算法基本原理。
- 然后引入主题模型概念，详细介绍了LSA算法和LDA算法的流程。最后根据3种关键词提取算法原理，编写每种算法自定义函数，并通过实例实现关键词提取。



THANK YOU

